



SAPIENZA
UNIVERSITÀ DI ROMA

Progettazione di sistemi digitali

UNIVERSITÀ DEGLI STUDI DI ROMA "LA SAPIENZA"

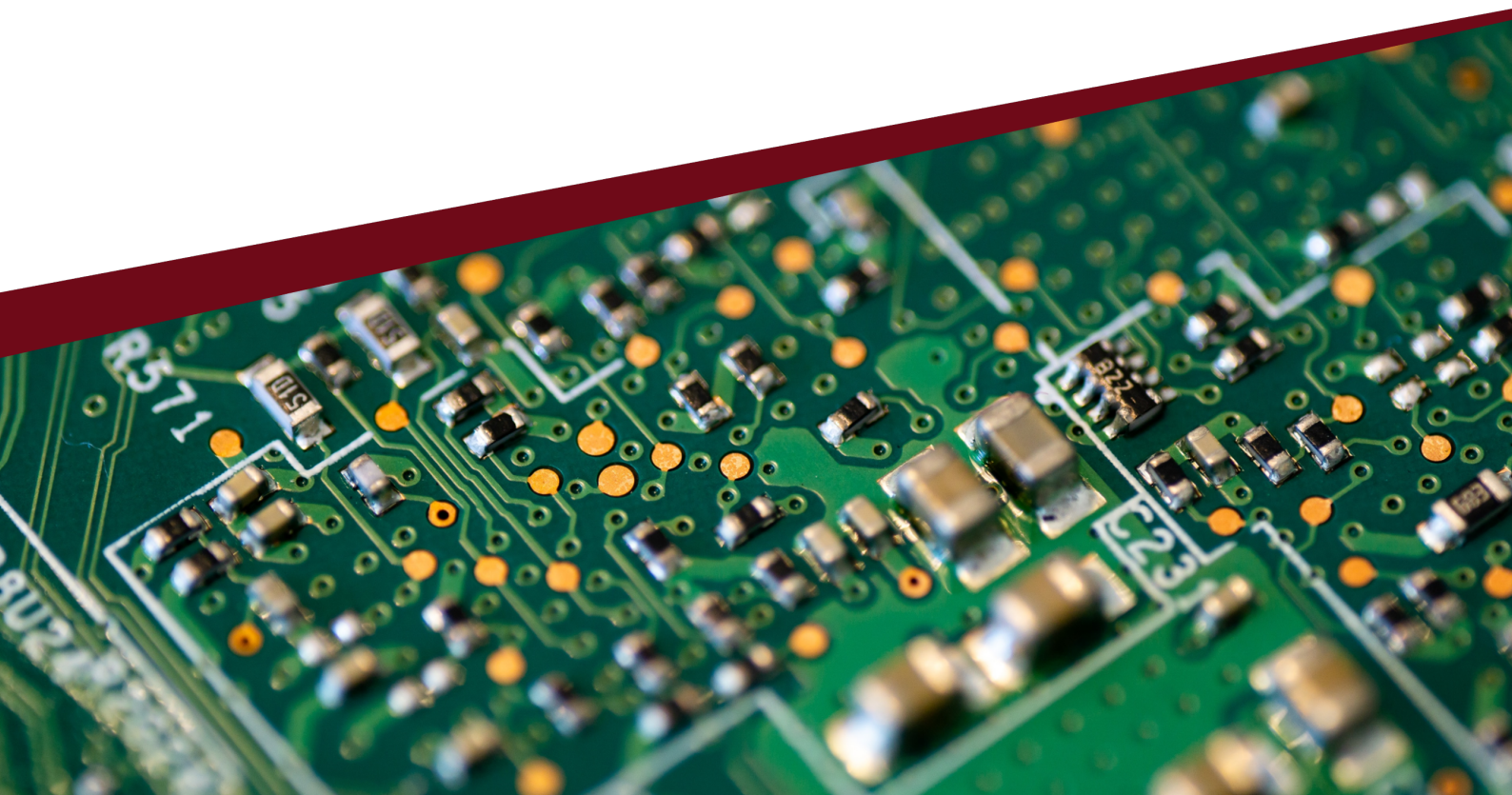
FACOLTÀ DI INGEGNERIA DELL'INFORMAZIONE, INFORMATICA E
STATISTICA

CORSO DI LAUREA TRIENNALE IN INFORMATICA

Autore
LORENZO SABATINO

Basato sulle lezioni di
PROF. ANNALISA MASSINI

19 ottobre 2024



Prefazione

Salve a tutti!

Mi chiamo Lorenzo Sabatino e sono uno studente del corso di Informatica presso l'Università La Sapienza di Roma. Durante il mio percorso accademico, ho affrontato numerosi esami trascrivendo gli appunti presi a lezione e trasformandoli in veri e propri libri di studio.

Ho voluto condividere questo materiale con voi perché credo che possa essere un valido aiuto per tutti gli studenti di Informatica, sia quelli alle prime armi che chi è già avanti nel percorso di studi. Ho scritto e formattato questi testi con cura, utilizzando il linguaggio di markup $\text{\LaTeX}$ per garantire chiarezza e leggibilità. Ho anche arricchito il contenuto con diagrammi e schemi e ho integrato spunti tratti da lezioni online e da libri di testo.

Tuttavia, è importante sottolineare che questi libri non sostituiscono le lezioni frontali e non sono privi di errori. La mia intenzione è stata quella di fornire un supporto aggiuntivo allo studio, ma è indispensabile partecipare alle lezioni e approfondire i concetti con la guida dei docenti. Inoltre, tenete presente che, data la natura in continua evoluzione dell'informatica, alcune informazioni potrebbero diventare obsolete col passare del tempo.

Nonostante ciò, ho deciso di condividere questi libri perché credo che possano essere d'aiuto a molti studenti e spero che li troviate utili per il vostro percorso accademico.

Se desiderate contattarmi per qualsiasi domanda, commento o correzione, potete scrivere all'indirizzo email scrittinformatica@gmail.com. Sarò lieto di rispondere alle vostre richieste.

Grazie per aver scelto di leggere questi libri e in bocca al lupo per i vostri studi!

Indice

Prefazione	1
1 Introduzione al corso	5
2 Rappresentare l'informazione	7
2.1 Rappresentazione numerica	7
2.2 Operazioni in binario	8
2.3 Rappresentazione dei numeri interi	9
2.3.1 Modulo e segno	9
2.3.2 Complemento a due	9
2.4 Rappresentazione dei numeri decimali	11
2.4.1 Virgola fissa	11
2.4.2 Virgola mobile (floating point)	11
3 Circuiti combinatori e algebra di Boole	15
3.1 Algebra di Boole	15
3.1.1 Assiomi e proprietà	15
3.2 Espressioni booleane	17
3.2.1 Trasformazione o semplificazione di espressioni booleane	17
3.2.2 Espressione complementare	17
3.2.3 Espressioni in forma normale SOP/POS	18
3.2.4 Canonica SOP/POS	18
3.2.5 Altri operatori	19
3.2.6 Minimizzazione	21
3.3 Reti combinatorie	24
3.4 Esercizi svolti	26
4 Blocchi costruttivi digitali	29
4.1 Circuiti aritmetici	29
4.1.1 Addizione	29
4.1.2 Sottrazione	31
4.1.3 Comparatori	31
4.2 Moduli notevoli	31
4.2.1 Decodificatore	31
4.2.2 Codificatore	32
4.2.3 Transcodificatore	33
4.2.4 Multiplexer	33
4.2.5 Demultiplexer	34
4.2.6 ROM	34
4.2.7 PLA	35
4.3 Esercizi svolti	37
5 Reti sequenziali	39
5.1 Latch SR	39
5.2 Flip-flop D	40
5.3 Flip-flop JK	41
5.4 Flip-flop T	41

5.5	Analisi e sintesi di una rete sequenziale	42
5.6	Esercizi svolti	47
6	Registri	55
6.1	Contatori	55
6.2	Registri	57
6.3	Trasferimento tra registri	60
6.3.1	Progettare uno schema di connessione tra registri	63
6.4	Esercizi svolti	64
	Link utili	67

Capitolo 1

Introduzione al corso

L'obiettivo del corso "Progettazione di sistemi digitali" è quello di studiare le tecniche per progettare circuiti all'interno del calcolatore o computer. Il seguito del corso interesserà infatti le cosiddette architetture degli elaboratori.

Il corso sarà inoltre suddiviso nelle seguenti parti:

- Rappresentazione dell'informazione;
- Circuiti combinatori e algebra di Boole;
- Moduli standard.

A questo punto abbiamo tutti gli strumenti necessari per creare un calcolatore tranne quelli della memoria. Quindi al corso si aggiungeranno:

- Circuiti sequenziali;
- Automi a stadi finiti;
- Registri;
- Interconnettere moduli e registri.

Prima di capire le funzionalità dei circuiti all'interno di un calcolatore, cos'è un calcolatore? Il modo più semplice per rispondere a questa domanda è utilizzando l'architettura di Von Neumann¹, secondo cui un calcolatore è formato da tre moduli fondamentali come mostrato in figura 1.1.

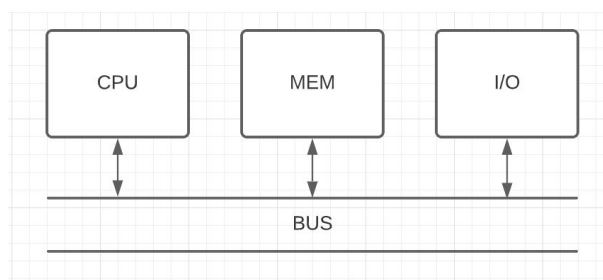


Figura 1.1: Schematizzazione del modello Von Neumann

- La CPU (Central Processing Unit o processore) è formata da ALU (Unità Aritmetica Logica) che contiene tutti i circuiti per eseguire le operazioni aritmetico-logiche e la CU (o unità di controllo) che controlla l'esecuzione dei programmi;
- Nell'architettura dei calcolatori, si distinguono due tipi di memoria: la memoria centrale primaria, che lavora a più diretto contatto con il processore e la memoria secondaria o di massa;

¹John von Neumann è stato un matematico, fisico e informatico ungherese, è considerato uno dei più grandi matematici della storia moderna

- I dispositivi di Input/Output sono invece tutti quei dispositivi che non appartengono alla CPU o alla memoria;
- La CPU, la memoria e i dispositivi di Input/Output si scambiano i dati attraverso il Bus. Il Bus è l'insieme di linee su cui viaggiano i segnali elettrici formati da 0 e 1.

Capitolo 2

Rappresentare l'informazione

Per rappresentare le informazioni si utilizzano un insieme di simboli utili al nostro scopo. Nel caso degli elaboratori questi simboli sono le famosissime cifre 0 e 1.

I = insieme di informazioni Σ = insieme di simboli C = insieme di parole

- Funzione di codifica $[C : I \rightarrow C]$;

E' proprio questo il processo attraverso cui il nostro elaboratore codifica, con un certo numero di bit, l'informazione. Un esempio di codifica che si verifica ogni giorno avviene quando digitiamo un carattere sulla tastiera del nostro computer.

Esempio 1. Abbiamo un insieme composto da vari elementi a cui fanno riferimento delle combinazioni di bit 0 e 1.

$I = \{\text{mela, pera, arancia, susina}\}$ $\Sigma = \{0, 1\}$ $C = \{00, 01, 10, 11\}$

- Funzione di decodifica $[D : C \rightarrow F]$.

I requisiti della codifica/decodifica sono:

- Insiemi devono essere finiti;
- $|I| = |C|$ codifica efficiente;
- $|I| > |C|$ codifica ambigua;
- $|I| < |C|$ codifica rindondante;
- Usare il minimo numero di simboli che mi permettono di rappresentare l'informazione, come nell'esempio 1;
- Facilit  di esecuzione della codifica/decodifica;
- Facilit  di esecuzione delle operazioni.

2.1 Rappresentazione numerica

Quando si parla della rappresentazione delle informazioni, e in particolar modo con i numeri naturali, l'informazione o numero da rappresentare   fortemente dipendente dal sistema di riferimento.

Con il sistema posizionale ogni cifra assume un valore in base alla sua posizione, tra questi rientrano il sistema decimale e gli altrettanti importanti sistemi ottale ed esadecimale. Tramite questo sistema posso rappresentare b^n valori (b = base, n = numero di cifre o bit), registrando le cifre comprese tra $\{0, \dots, b^n - 1\}$. La regola generale da seguire   quindi:

$$N = \sum_{i=0}^{n-1} C_i * b^i \quad (2.1)$$

Base	Numero	Conversione	Risultato
base 10	375	$3x10^2 + 7x10^1 + 5x10^0$	375
base 8	375	$3x8^2 + 7x8^1 + 5x8^0$	253
base 2	1011	$1x2^3 + 0x2^2 + 1x2^1 + 1x2^0$	11

Nel sistema ottale posso rappresentare i valori tra $\{0, \dots, 7\}$, divido quindi la cifra in blocchi 3 a 3 e parto dalla LSB¹.

Base	Numero
base 2	101.010.001.011
base 8	5.2.1.3

Nel sistema esadecimale posso rappresentare i valori $\{0, \dots, 9, A, \dots, F\}$, divido quindi la cifra in blocchi 4 a 4 e parto da LSB.

base 2	1010.1000.1011
base 16	A.8.B

Per convertire i numeri da decimale a binario si applica la divisione in base b iterata sul quoziente, dove la rappresentazione è data dalla sequenza dei resti presi in ordine inverso.

14		
7		0
3		1
1		1
0		1

$14_{(10)} = 1110_{(2)}$

A questo punto possiamo chiederci *quanti bit mi serviranno per rappresentare un dato valore?* Applicando la regola con il logaritmo avremo la risposta alla nostra domanda.

$$n = \log_b N \approx \text{per eccesso}$$

$$N = 1300 \quad n = \log_2 1300 = 11$$

Prima di passare al prossimo argomento ci sono delle micro informazioni da tenere a mente.

1 Byte	8 bit	
$2^{10} \simeq 10^3$	Kilo	KB
$2^{20} \simeq 10^6$	Mega	MB
$2^{30} \simeq 10^9$	Giga	GB

2.2 Operazioni in binario

Addizione

Per addizionare due numeri in binario si parte da LSB e si sommano le cifre nella stessa posizione in base 2.

¹ 0	¹ 1	1
0	0	1
1	0	0

Sottrazione

La sottrazione consiste nel sottrarre le cifre nella stessa posizione solo se il primo operando è maggiore del secondo.

0	1	1	0	⁰ 1	¹ 0	⁰ 1	¹ 0	¹ 0
0	0	1	0	0	1	0	0	1
0	1	0	0	0	1	0	1	1

¹Least Significant Bit o cifra meno significativa

Moltiplicazione

La moltiplicazione viene rappresentata come una semplice moltiplicazione tra cifre con l'accortezza che, moltiplicando due numeri binari, il risultato avrà bisogno di più bit. Generalmente il doppio. $[0, \dots, 2^n - 1]$

$$\begin{array}{r} 010 \\ 011 \\ \hline 010 \\ 011 \\ \hline 0110 \\ 0110 \end{array} \quad \begin{array}{r} 1111 \\ 0111 \\ \hline 1111 \\ 1111 \\ \hline 1101001 \end{array}$$

Overflow

L'overflow² indica un evento che si verifica quando non posso rappresentare in bit il risultato.

$$\begin{array}{r} 11101 \\ 0111 \\ \hline \bullet 1000 \end{array}$$

2.3 Rappresentazione dei numeri interi

Tutto chiaro fin qui? Bene! Ma se voglio rappresentare anche i numeri negativi in base 2?

Per rappresentare tutti gli interi si usano due diversi tipi di rappresentazione: il **Modulo e segno** e il **Complemento a due** che è il più utilizzato.

2.3.1 Modulo e segno

Utilizzando il sistema modulo e segno, si assegna ad MSB³ il valore 0 se il segno è positivo, il valore 1 se il segno è negativo.

$$1011 = -3$$

2.3.2 Complemento a due

Utilizzando il sistema complemento a due, si assegna sempre a MSB il segno meno.

$$-a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i * 2^i \quad (2.2)$$

Binario	Decimale	MeS	Ca2
000	0	0	0
001	1	1	1
010	2	2	2
011	3	3	3
100	4	-0	-4
101	5	-1	-3
110	6	-2	-2
111	7	-3	-1

Proprietà del complemento a due

Come già accennato prima il complemento a due è il sistema più usato. *Ma perchè è il più usato se il più "semplice" è modulo e segno?* Ebbene, il complemento a due è il sistema più efficiente, con il minor numero di operazioni da svolgere e il più rapido nel calcolare l'opposto di un numero. In caso contrario, ovvero adoperando il sistema modulo e segno, si costringerebbe al calcolatore di eseguire un'operazione di verifica in più prima di svolgere la codifica.

²In informatica, elaborazione eccedente il limite della capacità di una memoria.

³Most Significant Bit o cifra più significativa

Opposto di un numero

L'opposto di un numero si calcola semplicemente negando tutti i bit della rappresentazione e infine sommando 1.

$$0110 = 6 \quad \text{opposto} \quad \begin{array}{cccc} 1 & 0 & 10 & 1 \\ 0 & 0 & 0 & 1 \\ \hline 1 & 0 & 1 & 0 \end{array} = -6$$

Estensione di un numero

I numeri binari rappresentati in complemento a due sono potenzialmente estendibili all'infinito, dove per estensione si intende l'aggiunta di MSB.

$$\begin{aligned} 0101 &= 5 = 0000101 = 5 \\ 1011 &= -5 = 1111011 = -5 \end{aligned}$$

Codifica in complemento a due

Ok quindi posso estendere all'infinito il mio numero ma di quanti bit ho veramente bisogno? Per rappresentare un numero in complemento a due servono:

$$\log_2(|N|) + 1 \quad (2.3)$$

$$-10 = \log_2(10) + 1 = 4 + 1 = 5$$

Rappresentazione in complemento a due

Arrivati a questo punto, con le conoscenze apprese, non sarà molto difficile rappresentare un numero come complemento a due. Basta infatti trasformare il numero in binario, estendere un bit, calcolare l'opposto e sommare 1.

$$\begin{array}{c|cccccc} 10 & & & & & & & & & & \\ 5 & 0 & & 0 & 1 & 0 & 1 & 0 & & 1 & 0 & 1 & 10 & 1 \\ 2 & 1 & & 1 & 0 & 1 & 0 & 1 & & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & & & & & & & & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & & & & & & & & & & & & \end{array}$$

Addizione e sottrazione

La somma funziona sempre a patto che il risultato possa essere contenuto nell'intervallo. Ad esempio, la somma tra un numero positivo e uno negativo genera il risultato corretto.

Si noti inoltre che la somma funziona, a patto di trascurare l'ultimo bit di riporto.

		Risultati corretti.			
$\begin{array}{cccc} 1 & 11 & 10 & 11 & 11 \\ & 0 & 1 & 0 & 1 \\ \hline & 0 & 0 & 0 & 0 \end{array}$	$-5 + 5 = 0$	$\begin{array}{cccc} 1 & 10 & 1 & 1 & 1 \\ & 1 & 1 & 0 & 0 \\ \hline & 0 & 0 & 1 & 1 \end{array}$	$+7 - 4 = 3$		
		Risultati non corretti.			
$\begin{array}{cccc} 10 & 11 & 10 & 11 \\ & 0 & 1 & 1 & 1 \\ \hline & 1 & 1 & 0 & 0 \end{array}$	$+5 + 7 = -12$	$\begin{array}{cccc} 1 & 11 & 1 & 0 & 0 \\ & 1 & 0 & 1 & 1 \\ \hline & 0 & 1 & 1 & 1 \end{array}$	$-4 - 5 = 9$		

Per la sottrazione, invece, si calcola l'opposto del secondo operando e si esegue la somma.

$$6 - 3 = \begin{array}{cccc} & 0 & 1 & 1 & 0 \\ & 1 & 1 & 0 & 1 \\ \hline & 0 & 0 & 1 & 1 \end{array}$$

N° bit	Nome	Bit di segno	Esponente	Mantissa
16 bit	Half precision	1	5	10
32 bit	Single precision	1	8	23
64 bit	Double precision	1	11	52

Tabella 2.1: Tabella di conversione in floating point

Dunque per rappresentare in virgola mobile un numero decimale si parte dal fatto che un numero binario è scomponibile come segue:

$$(-1)^s * 1, m * 2^e \quad (2.4)$$

Partendo da questa nozione possiamo seguire vari passaggi:

- Trovo parte intera (P.I.) e parte frazionaria (P.F.) come se fosse un numero in virgola fissa;
- Riscrivo il numero partendo dall'1 più significativo inserendo una virgola subito dopo e moltiplicando per la potenza del 2 con l'esponente relativo allo shift effettuato;
- Uso quanto ottenuto per trovare M ed E;
- Applico le formule previste dalla regola 2.4;
 - **Segno** (s) = 0 per i numeri positivi, 1 per quelli negativi;
 - **Esponente** è dato dalla somma dell'esponente e il BIAS, $(e) = E + 2^{N-1} - 1$. Si converte il risultato in binario.
 - **Mantissa** (m) è data da $M = 1, m$;
- Infine compongo il risultato aggregando le parti in un'unica stringa: $\langle s, e, m \rangle$.

Esempio 2. Come rappresento in virgola mobile, single precision, il numero decimale $13,25_{10}$?

$$\begin{array}{r|l}
 \text{Parte intera: } 1101_2 & \text{Parte frazionaria: } 1_2 \\
 13 & \\
 6 & 1 \\
 3 & 0 \\
 1 & 1 \\
 0 & 1
 \end{array}
 \quad
 \begin{array}{r|l}
 0,25*2 & 0,5 & 0 \\
 0,5*2 & 1,0 & 1
 \end{array}
 \quad
 13,25 = 1101,01$$

Lo riscrivo partendo dall'1 più significativo, piazzandogli una virgola subito dopo e moltiplicandolo per la potenza del 2 con l'esponente relativo allo shift effettuato:

$$1101,01 = 1,10101 * 2^3$$

A questo punto ho trovato $M = 110101$ ed $E = 3$. Applico le formule per ricondurmi alla mantissa e all'esponente:

$$\begin{aligned}
 M = 1, m &\Rightarrow m = 10101 \\
 e = E + 2^{n-1} - 1 &\Rightarrow e = 3 + 2^7 - 1 = 3 + 127 = 130 \\
 \text{che in binario è: } &10000010. \text{ (}^n \text{ è il numero di cifre riportate nella tabella 2.1)}
 \end{aligned}$$

$s = 0$ perché il numero è positivo.

Risultato finale:

$$\begin{array}{c|c|c}
 s & e & m \\
 \hline
 0 & 10000010 & 10101\ 00000000000000000000 \\
 \hline
 < 0;100 & 0001 & 0;101\ 0100\ 0000\ 0000\ 0000\ 0000 > \\
 \hline
 4 & 1 & 5\ 4\ 0\ 0\ 0\ 0
 \end{array}$$

Conversione in esadecimale

Addizione e sottrazione in virgola mobile

L'**addizione** di numeri in virgola mobile non è semplice come l'addizione tra numeri in complemento a due. I passaggi per sommare due numeri in virgola mobile sono:

1. Estrarre i bit dell'esponente e quelli della mantissa dei due numeri;
2. Aggiungere l'1 più significativo per ottenere la mantissa effettiva;
3. Confrontare gli esponenti. In particolar modo, gli esponenti devono essere gli stessi. Se ciò non è già verificato bisogna sottrarre l'esponente minore dall'esponente maggiore. Il risultato è il numero di bit per il quale il numero minore deve essere traslato a destra affinché gli esponenti siano uguali;
4. Traslare se necessario la mantissa con esponente minore;
5. Sommare le due mantisse;
6. Normalizzare⁴ la mantissa risultante sistemando, se necessario, l'esponente;
7. Arrotondare il risultato se necessario;
8. Identificare il segno in base alla grandezza degli operandi sommati;
9. Assemblare segno, esponente e mantissa nella notazione in virgola mobile.

Capiamo meglio quanto scritto con un esempio.

Esempio 3 (Addizione). *Sommare in virgola mobile i numeri 7.875 e 0.1875.*

Dopo aver applicato i procedimenti descritti nella Sezione 2.4.2, rappresento i due numeri:

	0	10000001	111 1100 0000 0000 0000 0000
	0	01111100	100 0000 0000 0000 0000 0000
1.	0	10000001	111 1100 0000 0000 0000 0000
	0	01111100	100 0000 0000 0000 0000 0000
	Esponente		Mantissa
2.	10000001	1.111 1100 0000 0000 0000 0000	
	01111100	1.100 0000 0000 0000 0000 0000	
3.	10000001	1.111 1100 0000 0000 0000 0000	
	01111100	1.100 0000 0000 0000 0000 0000	
4.	10000001	1.111 1100 0000 0000 0000 0000	
	01111100	0.000 0110 0000 0000 0000 0000 0000	
5.	10.000 0010 0000 0000 0000 0000		

Dal momento che la somma ha una mantissa che è maggiore o uguale a 2.0, il risultato viene normalizzato traslandolo verso destra di un bit e aumentando l'esponente.

6.	100000 10	1.000 0001 0000 0000 0000 0000
7.	(non serve arrotondare)	
8.	entrambi numeri positivi, segno = 0	

Il risultato viene quindi immagazzinato in notazione in virgola mobile rimuovendo l'uno più significativo implicito della mantissa e impostando il bit di segno.

9.	0 10000010	000 0001 0000 0000 0000 0000
----	------------	------------------------------

⁴La normalizzazione è un procedimento che obbliga la traslazione dei bit verso destra se il numero in questione è maggiore o uguale a 2.0.

Esempio 4 (Sottrazione). Proviamo ora a svolgere la seguente sottrazione:

	0	11101	11101...0
	0	11110	11011...0
1.	0	11101	11101...0
	0	11110	11011...0
	Esponente	Mantissa	
2.	11101	1.11101...0	
	11110	1.11011...0	
	$e_r = 30 - 29 = 1$		
3.	11101	1.11101...0	
	11110	1.11011...0	
4.	11101	0.111101	
	11110	1.11011...0	
5.	B-A	1.110110- 0.111101	
		0.111001	
6.	1.11001x2 ²⁹		

Risultato:

9.	A-B	0	11101	11001...0
	B-A	1	11101	11001...0

Moltiplicazione e divisione in virgola mobile

Per quanto riguarda la moltiplicazione e divisione in virgola mobile, le operazioni da svolgere sono le seguenti:

1. Estrarre i bit dell'esponente e quelli della mantissa dei due numeri;
2. Aggiungere l'1 più significativo per ottenere la mantissa effettiva;
3. L'esponente è dato dalla somma o dalla differenza dei due operandi, meno il BIAS ed eventualmente normalizzato;
4. La mantissa è data dal prodotto o dal quoziente della mantissa dei due operandi, ed eventualmente normalizzata;
5. Il segno è pari a 0 se i segni dei due floating point sono concordi. In caso contrario il segno è pari a 1;
6. Assemblare segno, esponente e mantissa nella notazione in virgola mobile.

Esempio 5 (Moltiplicazione). $A = \langle 0; 11001; 01010...0 \rangle$ $B = \langle 0; 11000; 0110...0 \rangle$

1.	0	11001	01010...0
	0	11000	0110...0
2.	A	1,11001	
	B	1,11000	
3.	e_r	11001+11000	
		25+24-15(BIAS)	
		34	

Il numero 34 non è rappresentabile con i 5 bit dell'esponente quindi si presenta un Overflow.

Capitolo 3

Circuiti combinatori e algebra di Boole

3.1 Algebra di Boole

Partiamo dall'algebra perchè vogliamo costruire circuiti più efficienti possibili. In particolar modo all'algebra di commutazione perchè fa riferimento ai procedimenti dei calcolatori.

L'algebra di Boole è fomata da:

- Un alfabeto di supporto. $\Sigma = \{0, 1\}$;
- Da operazioni booleane quali:
 - **Not.** Dato a , not $a = \bar{a}$

a	$\bar{a}$
0	1
1	0

Figura 3.1: Tabella di verità del not

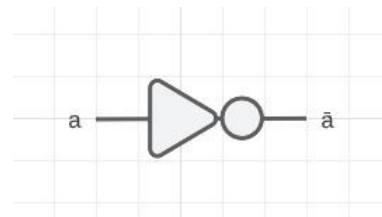


Figura 3.2: Porta logica del not

- **And.** Dati a e b , and $= a * b$

a, b	$a * b$
00	0
01	0
10	0
11	1

Figura 3.3: Tabella di verità dell'and

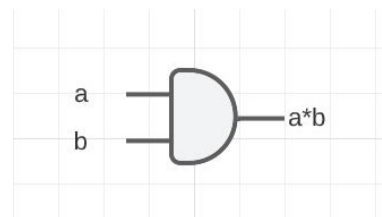


Figura 3.4: Porta logica dell'and

- **Or.** Dati a e b , or $= a + b$

3.1.1 Assiomi e proprietà

L'algebra booleana si basa su un insieme di postulati che come tali vengono per definizione considerati corretti. I postulati non sono dimostrabili, nel senso che un assunto di partenza non può essere

a, b	$a + b$
00	0
01	1
10	1
11	1

Figura 3.5: Tabella di verità dell'or

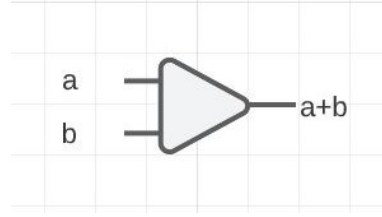


Figura 3.6: Porta logica dell'or

dimostrato. A partire da questi postulati è possibile invece dimostrare tutti i teoremi dell'algebra booleana.

Le proprietà e gli assiomi dell'algebra booleana obbediscono al principio di **dualità**: se i simboli 0 e 1 e gli operatori $*$ (AND) e $+$ (OR) sono scambiati tra loro, l'affermazione rimane corretta.

Assiomi

Definizione 1 (Associatività).

$$\begin{aligned} a + (b + c) &= (a + b) + c \\ a(b * c) &= (a * b)c \end{aligned} \quad (3.1)$$

Definizione 2 (Commutatività).

$$\begin{aligned} a + b &= b + a \\ a * b &= b * a \end{aligned} \quad (3.2)$$

Definizione 3 (Distributività).

$$\begin{aligned} a(b + c) &= a * b + a * c \\ a + b * c &= (a + b)(a + c) \end{aligned} \quad (3.3)$$

Definizione 4 (Elemento neutro).

$$\begin{aligned} a + 0 &= a \\ a * 1 &= a \end{aligned} \quad (3.4)$$

Definizione 5 (Complemento).

$$\begin{aligned} a + \bar{a} &= 1 \\ a * \bar{a} &= 0 \end{aligned} \quad (3.5)$$

Proprietà

Definizione 6 (Involuzione).

$$\bar{\bar{a}} = a \quad (3.6)$$

Definizione 7 (Idempotenza).

$$\begin{aligned} a + a &= a \\ a * a &= a \end{aligned} \quad (3.7)$$

Definizione 8 (Elemento nullo).

$$\begin{aligned} a + 1 &= 1 \\ a * 0 &= 0 \end{aligned} \quad (3.8)$$

Definizione 9 (Assorbimento 1).

$$\begin{aligned} a + ab &= a \\ a(a + b) &= a \end{aligned} \quad (3.9)$$

Definizione 10 (Assorbimento 2).

$$\begin{aligned} a + \bar{a}b &= a + b \\ a(\bar{a} + b) &= ab \end{aligned} \quad (3.10)$$

Definizione 11 (Leggi di Morgan).

$$\begin{aligned}\overline{a + b} &= \bar{a} * \bar{b} \\ \overline{a * b} &= \bar{a} + \bar{b}\end{aligned}\quad (3.11)$$

N.B. Assiomi e proprietà valgono anche per le espressioni booleane.

3.2 Espressioni booleane

- Un letterale è un'espressione booleana così come lo sono 0 e 1;
- Se E è un'espressione booleana, allora $\bar{E}$ è un'espressione booleana;
- Se E_1 e E_2 sono espressioni booleane, anche $E_1 + E_2$ ed $E_1 * E_2$ sono espressioni booleane;
- Sono espressioni booleane tutte e sole quelle che otteniamo con le espressioni precedenti.

3.2.1 Trasformazione o semplificazione di espressioni booleane

Esercizio 1. $a + b(\overline{c + d\bar{a}})$

Espressione	Proprietà
$a + b(\overline{c + d\bar{a}})$	De Morgan
$a + b(\bar{c}(\bar{d} + \bar{\bar{a}}))$	De Morgan + involuzione
$a + b(\bar{c}\bar{d} + \bar{a}\bar{c})$	Distributiva
$a + b\bar{c}\bar{d} + ab\bar{c}$	Distributiva
$a(1 + b\bar{c}) + b\bar{c}\bar{d}$	Raggruppamento + elemento nullo
$a + b\bar{c}\bar{d}$	Risultato

Verifica di identità

Esercizio 2. $ab + bc + \bar{a}c = ab + \bar{a}c$

Espressione	Proprietà
$ab + (a + \bar{a})bc + \bar{a}c$	Moltiplico per 1, $a + \bar{a}$
$ab + abc + \bar{a}bc + \bar{a}c$	Assorbimento
$ab + \bar{a}c$	Risultato

Se un'identità è vera, anche la sua duale è vera. Quindi:

$$(a + b)(b + c)(\bar{a} + c) = (a + b)(\bar{a} + c)$$

3.2.2 Espressione complementare

L'espressione complementare è un'identità che si ottiene negando l'espressione di partenza. Data E la sua complementare è $\bar{E}$.

$$E = ab + \bar{a}c \quad \bar{E} = \overline{ab + \bar{a}c}$$

Per risolverla: $\overline{ab + \bar{a}c} = (\bar{a} + \bar{b})(a + \bar{c})$

Riguardando l'esercizio 2 notiamo come i due risultati sono complementari. Quindi per ottenere la complementare di un'espressione booleana, posso applicare de Morgan come nell'esempio oppure passare alla duale e complementare i singoli letterali.

Quanto vale un'espressione booleana?

Il valore di un'espressione booleana si può ottenere tramite la tavola di verità.

Esercizio 3. $(a + b)(\bar{a} + c)$

Quest'espressione si può risolvere lasciandola così ovvero come prodotto di somme (POS) oppure calcolando la duale e quindi come somma di prodotti (SOP). In entrambe i casi la tavola di verità è la seguente:

abc	f
000	0
001	0
010	1
011	1
100	0
101	1
110	0
111	1

Tabella 3.1: Tavola di verità dell'esercizio 3

3.2.3 Espressioni in forma normale SOP/POS

Per trasformare espressioni booleane in forma normale SOP¹/POS², procediamo nel seguente ordine:

1. Applicare de Morgan;
2. Applicare la proprietà distributiva:
 - SOP, $a(b + c) = ab + ac$;
 - POS, $a + bc = (a + b)(a + c)$.
3. Eliminare termini rindondanti.

Esempio 6. $\overline{x_1} + x_2 + \overline{x_1} + x_3$ Forma SOP: $\overline{x_1}\overline{x_2} + \overline{x_1}x_3$ $(x_1\overline{x_2} + \overline{x_1})(x_1\overline{x_2} + x_3)$ $(x_1 + \overline{x_1})(\overline{x_2} + \overline{x_1})(x_1 + x_3)(\overline{x_2} + x_3)$
 Forma POS: $1(\overline{x_1} + \overline{x_2})(x_1 + x_3)(\overline{x_2} + x_3)$

3.2.4 Canonica SOP/POS

Viene chiamata forma canonica l'espressione booleana dove tutti i fattori contengono tutti i letterati.

- SOP se tutti prodotti sono mintermini³;
- POS se tutte le somme sono maxtermini⁴.

Per ottenere la forma canonica da una forma normale si moltiplica per $x + \overline{x}$ nel caso della SOP, si somma per il prodotto $x\overline{x}$ nella POS, dove x rappresenta l'elemento mancante. In tutti e due i casi infine si applica la proprietà distributiva e si eliminano i termini rindondanti.

N.B. Le forme canoniche permettono di stendere le tavole di verità con più facilità.

Esempio 7 (SOP). Normale SOP: $a\overline{b} + \overline{a}c$
 La trasformo in forma canonica SOP: $\overline{a}\overline{b}(c + \overline{c}) + \overline{a}c(b + \overline{b})$
 $\overline{a}\overline{b}c + \overline{a}\overline{b}\overline{c} + \overline{a}bc + \overline{a}\overline{b}c$

Inserisco i valori ottenuti all'interno della tabella di verità.

Questo perchè il nostro obiettivo è quello di trovare i valori per cui il prodotto sia uguale a 1 (ovvero quando tutti i termini valgono 1).

¹Sum Of Products, Somma di prodotti

²Products Of Sum, Prodotto di somme

³Ogni prodotto contiene tutti i letterali.

⁴Ogni somma contiene tutti i letterali.

a, b, c	g
000	0
001	1
010	0
011	1
100	1
101	1
110	0
111	0

Identifico il prodotto delle equazioni con 1 nel letterale e 0 nella forma complementata.

Es. $\bar{a}\bar{b}c$ vale
 $a = 1, \bar{b} = 0, c = 1$ e inserisco 1 nella tabella in corrispondenza del valore 101.

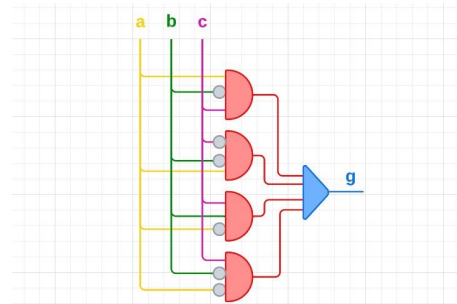
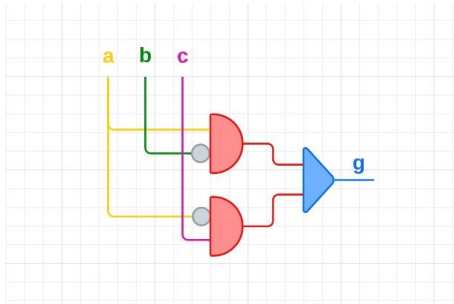


Figura 3.7: Circuiti SOP in forma normale e canonica

Esempio 8 (POS). Normale POS: $(\bar{a} + b + c)(a + \bar{b})$

La trasformo in canonica POS: $(\bar{a} + b + c)(a + \bar{b} + c\bar{c})$

$(\bar{a} + b + c)(a + \bar{b} + c)(a + \bar{b} + \bar{c})$

Inserisco i valori ottenuti all'interno della tabella di verità.

a, b, c	h
000	1
001	1
010	0
011	0
100	0
101	1
110	1
111	1

In POS ragioniamo in maniera duale: identifico quale somma di letterali riulterà 0.

Es. $\bar{a} + b + c$ vale
 $\bar{a} = 1, b = 0, c = 0$ e inserisco 0 nella tabella in corrispondenza del valore 100.

Questo perchè vogliamo trovare i valori per cui la somma sia uguale a 0 (ovvero quando tutti i termini valgono 0).

In questo modo, con la tabella di verità, è molto facile passare da SOP a POS e viceversa.

La SOP canonica di g sarà data dai valori corrispondenti a 1 nella tabella h . Mentre la POS canonica di g sarà data dai valori corrispondenti a 0 nella tabella g .

Infine, l'importanza della SOP e della POS è collegata all'utilità nei circuiti interessati. La SOP è formata da porte AND che confluiscono in OR. La POS da porte OR che confluiscono in AND.

3.2.5 Altri operatori

Oltre agli operatori fondamentali (AND, OR, NOT) esistono altri operatori che ci permettono di semplificare i nostri circuiti.

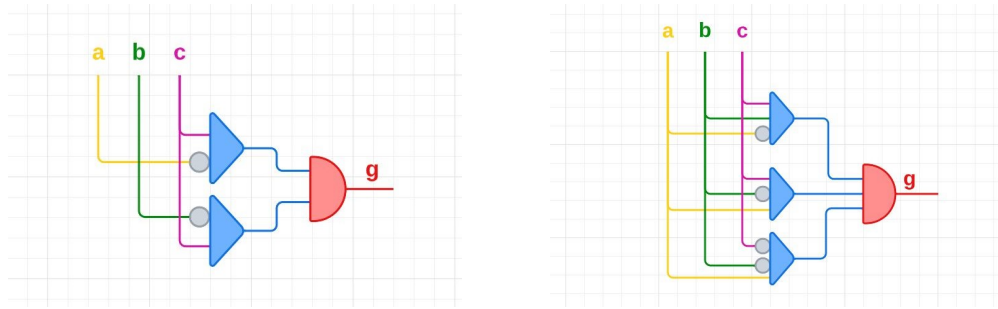


Figura 3.8: Circuiti POS in forma normale e canonica

XOR o OR ESCLUSIVO

L'operatore XOR vale 1 solo se uno dei due ingressi vale 1, altrimenti 0.

a, b	$a \oplus b$
00	0
01	1
10	1
11	0

Figura 3.9: Tabella di verità dello xor

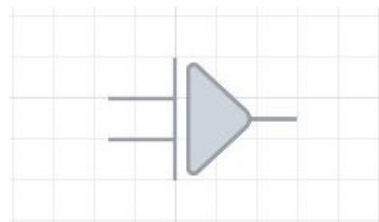


Figura 3.10: Porta logica dello xor

Forma canonica SOP
 $\bar{a}b + a\bar{b}$

Forma canonica POS
 $(a + b)(\bar{a} + \bar{b})$

NAND

La NAND (NOT-AND) è la complementare della AND e vale 1 solo se il risultato del prodotto degli ingressi è uguale a 0.

a, b	$\overline{ab}$
00	1
01	1
10	1
11	0

Figura 3.11: Tabella di verità della NAND

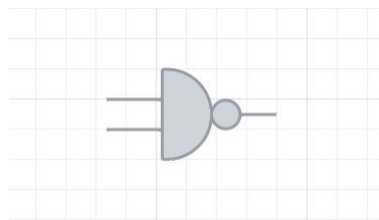


Figura 3.12: Porta logica della NAND

Come vedremo successivamente è possibile creare formule SOP e POS con sole NAND.

SOP con sole NAND

E' conveniente perchè si mantiene il sistema di un'operazione SOP ma utilizzando sempre la stessa porta. A livello realizzativo è più semplice per il calcolatore.

POS con sole NAND

Non è conveniente perchè aumenta solo il numero di operatori utilizzati.

Operazioni con il NAND:

$$\begin{aligned}\bar{a} &\equiv \overline{\overline{a}} \\ ab &= \overline{\overline{ab}} = \overline{\overline{a}\overline{b}} \\ a + b &= \overline{\overline{a+b}} = \overline{\overline{a}\overline{b}}\end{aligned}$$

NOR

Il NOR (NOT-OR) è il complementare dell'OR e vale 1 solo se il risultato della somma degli ingressi è uguale a 0.

a, b	$\overline{a + b}$
00	1
01	0
10	0
11	0

Figura 3.13: Tabella di verità del NOR

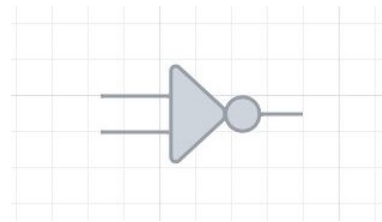


Figura 3.14: Porta logica del NOR

Anche in questo caso è possibile creare formule SOP e POS con soli NOR.

SOP con sole NOR

Non è conveniente perchè aumenta solo il numero di operatori utilizzati.

POS con sole NOR

È conveniente perchè si mantiene il sistema di un'operazione POS ma utilizzando sempre la stessa porta.

Operazioni con il NOR:

$$\begin{aligned}\bar{a} &= \overline{a + a} \\ ab &= \overline{\overline{a + b}} = \overline{\overline{a + a} + \overline{b + b}} \\ a + b &= \overline{\overline{a + b}} = \overline{\overline{a + b} + \overline{a + b}}\end{aligned}$$

Osservazione 1 (Se ho più di due ingressi cosa succede?). *Il risultato è lo stesso della forma normale ma il procedimento per raggiungerlo cambia. Perchè con il NAND e NOR si hanno più complementazioni che vengono semplificate.*

Per approfondire vedi Esercizio 4.

Ora risulta più chiara l'importanza delle porte NAND e NOR: queste porte possiedono la proprietà di completezza funzionale. Vuol dire che ogni altra funzione logica (AND, OR, NOT, etc.) può essere implementata utilizzando solamente porte NAND nella SOP, NOR nella POS, diventando così le uniche due porte logiche universali.

3.2.6 Minimizzazione

Quando possiamo definire che una rete è minimale?

Una rete AND-TO-OR/OR-TO-AND è minimale se:

- tra tutte le reti che realizzano la funzione data, la rete ha il minimo numero di porte AND/OR;
- tra tutte le reti con numero minimo di porte, la rete ha il minimo numero di ingressi per ogni porta.

Osservazione 2. *Potranno esistere più versioni minimali di una rete.*

Detto ciò, un'espressione SOP/POS è minimale se:

- tra tutte le espressioni SOP/POS che realizzano la funzione f , l'espressione ha il minimo numero di termini prodotto/somma;
- tra tutte le espressioni SOP/POS che realizzano i termini f , l'espressione ha il minimo numero di letterali per ogni termine.

Esistono varie tecniche per effettuare la minimizzazione. Quella che useremo sarà utilizzabile con massimo 4 variabili.

Tecnica delle mappe di Karnaugh (K-Mappe)

La tecnica delle mappe di Karnaugh permette di individuare i termini prodotto/somma adiacenti. Ovvero quando differiscono solo di una variabile (vera e complementata).

Esempio 9. $\bar{a}bc + a\bar{b}\bar{c}$

Notiamo che possiamo raccogliere $\bar{a}b$ essendo in comune in tutti e due i termini. Avremo quindi $\bar{a}b(c + \bar{c})$ che darà come risultato $\bar{a}b$.

Una **K-mappa** è una visualizzazione della tavola di verità come tabella a due ingressi. Le K-mappe si ottengono stendendo la tavola di verità per trovare i termini che differiscono per una variabile. Capiamo meglio con gli esempi che seguono:

Esempio 10.

$$\bar{a}bc + \bar{a}\bar{b}c + a\bar{b}\bar{c} + abc \quad (3.12)$$

Stendiamo la tavola di verità dell'equazione:

abc	f
000	0
001	1
010	0
011	0
100	1
101	1
110	0
111	1

Tabella 3.2: Tavola di verità dell'equazione 3.12

La K-mappa, a questo punto, si realizzerà con le colonne che fanno riferimento ai valori di a , mentre le righe a quelli di b e c . La tabella sarà quindi una matrice che conterrà i termini e valori della tabella di verità con l'unica differenza che le coppie bc saranno scritte come 00, 01, 11, 10 perchè le righe devono essere adiacenti (devono differire solo di uno 0 o 1).

$\begin{array}{c c} a & \\ \hline bc & \end{array}$	0	1
00	0	1
01	1	1
11	0	1
10	0	0

Tabella 3.3: K-mappa dell'espressione 3.12

Ora per scrivere l'espressione raccolgo a due a due i mintermini adiacenti (es. 100, 101). In questo modo mi trovo i valori che differiscono solo di una variabile. Il secondo passaggio è quello di scrivere l'espressione, guardando cosa hanno in comune i mintermini considerati secondo i valori di ab e cd nella tabella. Si scriverà quindi il letterale normale se in tabella corrisponde a 1, complementato se corrisponde a 0 (es. 001, 101 = $\bar{b}c$ oppure es. 101, 111 = ac). Come ultima cosa aggiungiamo i termini che non si trovano in adiacenza con nessun'altro termine. In questo modo il risultato sarà minimizzato.

SOP minimale: $\bar{a}b + \bar{b}c + ac$

Esempio 11. Cosa accade con 4 variabili?

Con 4 variabili si aggiunge alla coppia cd la coppia ab .

Rispetto all'esempio precedente cambia la grandezza della mappa (vedi Tabella 3.5). Per tanto come prima cosa dobbiamo trovare gli 1 nel cubo massimale. Questo vuol dire trovare 4 1 adiacenti disposti a quadrato o lungo una riga/colonna, per identificare le variabili comuni. Successivamente copriremo i casi degli 1 in doppio e infine i singoli 1 non adiacenti a nessun'altro 1. In questo modo possiamo trovare la SOP minimale.

SOP minimale: $ad + b\bar{c} + \bar{b}cd$

Per trovare la POS invece dobbiamo attuare lo stesso procedimento ma raccogliendo gli 0 e scrivendo la variabile in forma normale se vale 0, altrimenti complementata.

POS minimale: $(b + d)(\bar{c} + d)(a + b + c)(a + \bar{b} + \bar{c})$

$abcd$	f
0000	0
0001	0
0010	0
0011	1
0100	1
0101	1
0110	0
0111	0
1000	0
1001	1
1010	0
1011	1
1100	1
1101	1
1110	0
1111	1

Tabella 3.4: Tavola di verità

$\begin{smallmatrix} cd \\ ab \end{smallmatrix}$	00	01	11	10
00	0	0	1	0
01	1	1	0	0
11	1	1	1	0
10	0	1	1	0

Tabella 3.5: K-mappa

N.B. I quadrati e le coppie valgono anche per gli angoli della tabella o passando dalla prima riga/colonna all'ultima.

Tavole con valori indeterminati

Nelle tavole di verità possono esserci dei valori indeterminati chiamati *don't care*, rappresentati con δ o -. La presenza di questi valori è giustificata da due motivi:

1. non si possono rappresentare gli ingressi ($\bullet$);
2. non si possono rappresentare le uscite ($\times$).

Esempio 12. Dato x come valore compreso tra $[0;5]$, stendere la tavola di verità che rappresenta y compreso tra $[0;5]$.

$x_2x_1x_0$	$y_2y_1y_0$
000	010
001	011
010	100
011	101
100	011
101	100
110	--
111	--

Tabella 3.6: Tavola di verità con più output

Per trovare i minimali stendiamo le mappe di Karnaugh.

I - possono assumere il valore di cui ho bisogno, nella maniera più conveniente che mi permette di avere il cubo di valori più grande. In questo caso considero i *don't care* come 1.

$x_1x_0 \backslash x_2$	00	01	11	10
0	0	0	1	1
1	0	1	-	-

Tabella 3.7: Mappa per output y_2

SOP minimale: $x_1 + x_2x_0$

POS minimale: $(x_2 + x_1)(x_1 + x_0)$

Applico le stesse regole alle ultime due uscite y_1 e y_0 .

$x_1x_0 \backslash x_2$	00	01	11	10
0	1	1	0	0
1	1	0	-	-

Tabella 3.8: Mappa per output y_1

SOP minimale: $\overline{x_2x_1} + \overline{x_1x_0}$

POS minimale: $\overline{x_1}(\overline{x_2} + \overline{x_0})$

$x_1x_0 \backslash x_2$	00	01	11	10
0	0	1	1	0
1	1	0	-	-

Tabella 3.9: Mappa per output y_0

SOP minimale: $\overline{x_2x_0} + x_2\overline{x_0}$

POS minimale: $(x_2 + x_0)(x_2 + \overline{x_0})$

Infine per rappresentare la rete corrispondente posso scegliere le forme minimali più convenienti in termini di operatori utilizzati.

3.3 Reti combinatorie

Analizziamo ora le porte logiche (accennate nella sezione 3.1) più nel dettaglio.

Sono reti combinatorie:

- I terminali di ingresso;
- $\overline{N}$ se N è una rete combinatoria;
- $N_1 * N_2$ se N_1 e N_2 sono reti combinatorie;
- $N_1 + N_2$ se N_1 e N_2 sono reti combinatorie;
- Sono reti combinatorie tutte e sole le reti descritte sopra.

N.B. Con le reti combinatorie si possono formare anche dei cicli.

Stando a quanto detto finora, ad ogni espressione booleana corrisponde una rete combinatoria.

Esempio 13. $(x_1 + \overline{x_2} + x_3)x_1x_2\overline{x_3}$

Procedimento di sintesi di una rete combinatoria

Per sintetizzare una rete combinatoria partendo da un dato problema, i passi da seguire sono i seguenti:

- Leggere la specifica verbale o traccia del problema;

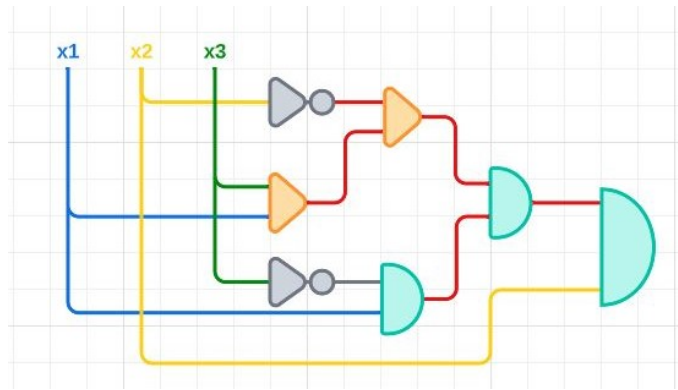


Figura 3.15: Esempio di rete combinatoria

- Ricavare l'espressione booleana (se necessario);
- Stendere la tavola di verità;
- Trovare le espressioni minimali con le K-Mappe;
- Disegnare la rete combinatoria;
- Analizzare (se richiesto).

Per approfondire vedi Esercizio 6.

Analisi di una rete combinatoria

Il procedimento opposto di quanto descritto sopra è formato dai seguenti passi:

- Partire dalla rete combinatoria data;
- Ricavare le espressioni booleane;
- Stendere la tavola di verità;
- Semplificare le espressioni (se necessario);
- Scrivere una descrizione verbale.

3.4 Esercizi svolti

Esercizio 4 (Costruire una funzione di maggioranza che vale 1 se almeno la metà delle variabili vale 1). Per costruire la funzione di maggioranza come prima cosa la rappresentiamo su un tabella di verità e identifichiamo con m_i i mintermini e con M_i i maxtermini.

	a, b, c	f
M_0	000	0
M_1	001	0
M_2	010	0
m_3	011	1
M_4	100	0
m_5	101	1
m_6	110	1
m_7	111	1

Tabella 3.10: Tabella di verità della funzione di maggioranza

Otteniamo ora le forme SOP e POS:

SOP: $m_3 + m_5 + m_6 + m_7 = \bar{a}bc + a\bar{b}c + ab\bar{c} + abc$

POS: $M_0 + M_1 + M_2 + M_4 = (a + b + c)(a + b + \bar{c})(a + \bar{b} + c)(\bar{a} + b + c)$

Se vogliamo rappresentare la funzione con sole NAND o soli NOR, moltiplichiamo il termine abc nella SOP e $a + b + c$ nella POS, tante volte quanti sono i termini che vogliamo semplificare.

SOP: $bc + ac + ab$

POS: $(a + b)(a + c)(b + c)$

Rappresentiamo ora i nostri risultati in ALL-NAND e ALL-NOR.

ALL-NAND: Partiamo dalla SOP e sostituiamo ogni operatore con una NAND.

$$f = \overline{\overline{bc} \overline{ac} \overline{ab}}$$

ALL-NOR: Partiamo dalla POS e sostituiamo ogni operatore con una NOR.

$$f = \overline{a + b + a + c + b + c}.$$

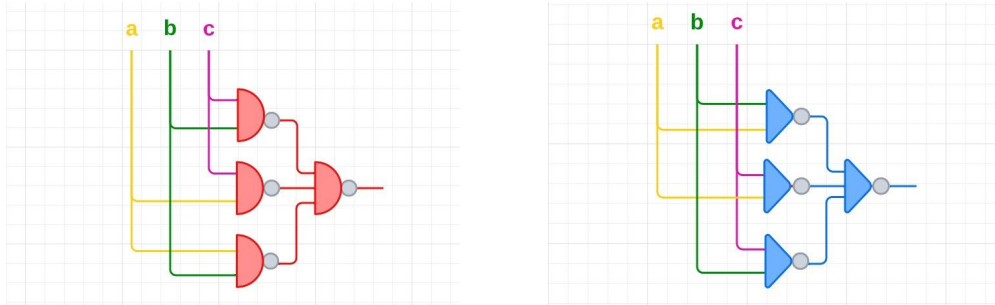


Figura 3.16: Circuiti in ALL-NAND e ALL-NOR

Esercizio 5. Data la funzione $f = (\bar{a}\bar{b}\bar{c})(bc + \bar{a}c)$ trovare la normale e canonica SOP e POS, tracciare la tabella di verità. Infine inserire nella tabella le funzioni $\tilde{f}$ (duale di f) e $\bar{f}$ (complementare di f).

Applichiamo de Morgan sulla funzione $(\bar{a}(\bar{b} + \bar{c}))(bc + \bar{a}c)$. Applichiamo ora la proprietà distributiva $(\bar{a}\bar{b} + \bar{a}\bar{c})(bc + \bar{a}c)$, svolgiamo le moltiplicazioni $\bar{a}\bar{b}bc + \bar{a}\bar{b}\bar{a}c$. Semplificando i termini opposti otteniamo $\bar{a}c + \bar{a}$ e infine per semplificare il primo termine moltiplichiamo per $\bar{b}\bar{c}$.

In questo modo otteniamo le forme normali SOP e POS (in questo caso sono le stesse): $\bar{a}\bar{b}\bar{c}$.

Per calcolare le forme canoniche proseguiamo nel seguente modo:

Canonica SOP: l'espressione è già in forma canonica SOP.

Canonica POS: $(\bar{a} + \bar{b}\bar{b} + \bar{c}\bar{c}) + (a\bar{a} + b + \bar{c}\bar{c})(a\bar{a} + \bar{b}\bar{b} + \bar{c})$. Trovo ora tutte le combinazioni per $\bar{a}$ fisso, b fisso e $\bar{c}$ fisso.

$$\bar{a}: (\bar{a} + b + c)(\bar{a}\bar{b} + c)(\bar{a} + b + \bar{c})(\bar{a} + \bar{b} + \bar{c})$$

$$b: (a + b + c)(a + b + \bar{c})(\bar{a} + b + c)(\bar{a} + b + \bar{c})$$

$$\bar{c}: (a + b + \bar{c})(a + \bar{b} + \bar{c})(\bar{a} + b + \bar{c})(\bar{a} + \bar{b} + c)$$

Dopo aver eliminato i termini ripetuti, rappresentiamo i valori ottenuti nella tabella di verità.

abc	f	$\tilde{f}$	$\bar{f}$
000	0	1	1
001	0	1	1
010	1	1	0
011	0	1	1
100	0	1	1
101	0	0	1
110	0	1	1
111	0	1	1

Tabella 3.11: Tavola di verità

Duale: $\bar{a} + b + \bar{c}$. Essendo POS, per la duale trovo i valori di 0 per cui è verificata (solo per 101).

Esercizio 6. Progettare un circuito che regola l'accensione di un'impianto di innaffiamento. L'impianto si attiva se si verificano **uno** dei tre eventi:

- Il sensore crepuscolare rileva che la luce è sotto una certa soglia;
- Il timer è impostato sull'ora di accensione e non piove;
- Non è l'ora di accensione ma la temperatura è sopra una certa soglia.

Come prima cosa dobbiamo definire le variabili di ingresso per la nostra funzione.

Ingresso:

$$\text{Sensore } A = \begin{cases} 1 \text{ luce sotto la soglia} \\ 0 \text{ altrimenti} \end{cases}$$

$$\text{Ora } O = \begin{cases} 1 \text{ se è giusta} \\ 0 \text{ altrimenti} \end{cases}$$

$$\text{Piove } P = \begin{cases} 1 \text{ se piove} \\ 0 \text{ altrimenti} \end{cases}$$

$$\text{Temperatura } T = \begin{cases} 1 \text{ se temperatura} > \text{ della soglia} \\ 0 \text{ altrimenti} \end{cases}$$

Uscita:

$$\text{Innaffiamento } INN = \begin{cases} 1 \text{ acceso} \\ 0 \text{ spento} \end{cases}$$

Ricaviamo ora la nostra espressione:

$$A + O\bar{P} + \bar{O}T \quad (3.13)$$

Stendiamo la tavola di verità.

Per verificare se l'espressione 3.13 è la migliore utilizziamo le K-Mappe.

Da qui possiamo ricavare le espressioni minimali SOP e POS.

SOP minimale: $A + O\bar{P} + \bar{O}T$

POS minimale: $(A + \bar{O} + \bar{P})(A + O + T)$

$AOPT$	f
0000	0
0001	1
0010	0
0011	1
0100	1
0101	1
0110	0
0111	0
1000	1
1001	1
1010	1
1011	1
1100	1
1101	1
1110	1
1111	1

Tabella 3.12: Tavola di verità

Capitolo 4

Blocchi costruttivi digitali

Nei capitoli precedenti è stata esaminata la progettazione delle reti combinatorie e sequenziali usando le espressioni booleane. In questo capitolo vengono introdotti blocchi costruttivi sia combinatori sia sequenziali più complessi, utilizzati nei sistemi digitali.

4.1 Circuiti aritmetici

I circuiti aritmetici sono i blocchi costruttivi centrali dei calcolatori. I calcolatori e la logica digitale eseguono molte funzioni aritmetiche come addizioni, sottrazioni, confronti, traslazioni, moltiplicazioni e divisioni. In questo paragrafo viene descritta la realizzazione hardware per ognuna di queste operazioni.

4.1.1 Addizione

L'addizione è una delle operazioni più comuni nei sistemi digitali. Inizieremo ad esaminare come sia possibile sommare due numeri binari a 1 bit. Per poi estendere l'operazione ai numeri binari ad N bit.

Half Adder

Si comincia dalla costruzione di un semisommatore (half adder) a 1 bit. Come mostrato in Figura

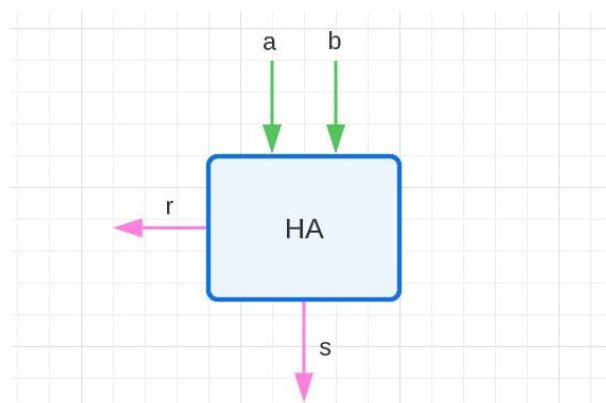


Figura 4.1: Semisommatore o Half Adder

4.1, il semisommatore ha due ingressi, A e B, e due uscite, S e R. S rappresenta la somma di A e B. Se A e B hanno valore 1, S è uguale a 2, un valore che non può essere rappresentato con una sola cifra binaria. Di conseguenza, il valore 2 viene rappresentato con un riporto (carry) R nella colonna successiva. Il semisommatore può essere costruito con una porta XOR e una porta AND.

abc	rs
000	00
001	01
010	01
011	10

Tabella 4.1: Tavola di verità dell'Half Adder

Full Adder

Un sommatore completo (full adder), è formato dalla concatenazione di più half adder e accetta in ingresso il valore c come mostrato nella Figura 4.2.

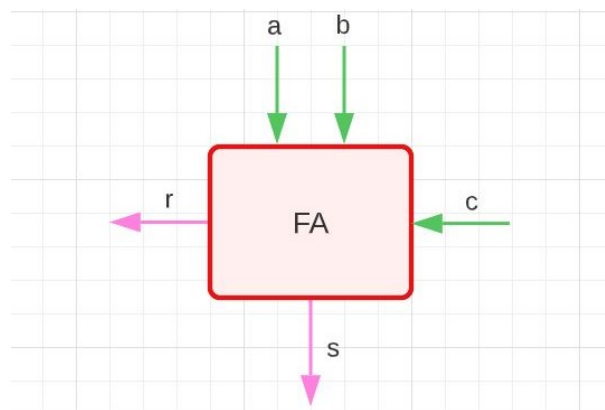


Figura 4.2: Sommatore completo o Full Adder

abc	rs
000	00
001	01
010	01
011	10
100	01
101	10
110	10
111	11

Tabella 4.2: Tavola di verità del Full Adder

Ripple Carry Adder

Un sommatore a N bit somma due ingressi a N bit: A e B. A questi aggiunge c, S per ottenere un risultato ad N bit e un riporto R. Questo sommatore viene comunemente chiamato sommatore a propagazione di riporto (o RCA, ripple carry adder) perché il riporto di un bit si propaga nel bit successivo.

Il metodo più semplice per costruire un sommatore a propagazione di riporto è collegare in cascata N full adder completi. In questo modo, R di uno stadio fa riferimento a c dello stadio successivo, come mostrato in Figura 4.3.

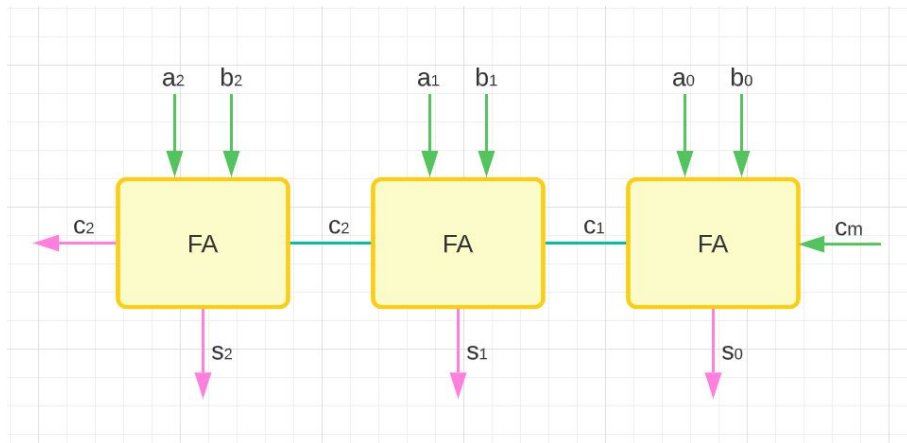


Figura 4.3: Sommatore a propagazione di riporto o Ripple Carry Adder

Notiamo che se l'ultima somma genera un riporto c , allora si produce overflow.

4.1.2 Sottrazione

I sommatore sono in grado di sommare numeri sia positivi sia negativi utilizzando la rappresentazione dei numeri in complemento a due. La sottrazione è quindi facile quanto l'addizione: si inverte il segno del secondo numero e poi si esegue la somma. Il cambio di segno di un numero in complemento a due si esegue negando tutti i bit e aggiungendo un 1.

Ad esempio se si vuole calcolare $Y = A - B$, per prima cosa si crea il numero in complemento a due di B : si negano tutti i bit di B per ottenere $\bar{B}$ e si aggiunge 1 per ottenere $-B = \bar{B} + 1$. Questo valore viene aggiunto ad A per ottenere $Y = A - B + 1 = A - B$. Questo risultato può essere ottenuto con l'utilizzo di un sommatore a propagazione di riporto facendo la somma $A + B$ con $c = 1$.

N.B. Per usare RCA con la sottrazione abbiamo bisogno di un segnale distinto che ci permette di specificare l'operazione da compiere (0: addizione, 1: sottrazione).

4.1.3 Comparatori

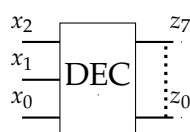
Un comparatore determina se due numeri binari sono uguali o se uno dei due è maggiore o minore dell'altro. Un comparatore riceve due numeri binari a N bit, A e B . La comparazione di valore dei numeri con segno viene solitamente effettuata calcolando $A - B$ e guardando il segno (cioè il bit più significativo) del risultato dell'operazione. Se il risultato è negativo (cioè se il bit del segno è uguale a 1) allora A è minore di B . Al contrario, se il risultato è positivo, A è maggiore o uguale a B .

4.2 Moduli notevoli

Vediamo ora come sono strutturati i principali moduli di un elaboratore. Questi moduli possono essere combinati tra di loro per svolgere in maniera più veloce le funzioni desiderate.

4.2.1 Decodificatore

Un decodificatore è un modulo formato da n ingressi e 2^n uscite, di cui solo una di queste vale 1.



Notiamo come la tavola del decodificatore 4.3 è facilmente realizzabile attraverso la diagonale di 1. Quindi le equazioni delle uscite saranno:

x_2	x_1	x_0	z_0	z_1	z_2	z_3	z_4	z_5	z_6	z_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

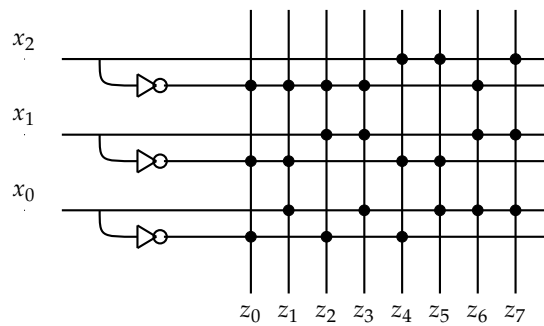
Tabella 4.3: Tavola del decodificatore

$$z_0 = \overline{x_2 x_1 x_0}$$

$$z_1 = \overline{x_2 x_1} x_0$$

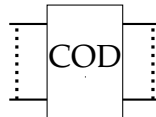
...

$$z_7 = x_2 x_1 x_0$$



4.2.2 Codificatore

Il codificatore è un modulo formato da n ingressi, di cui solo uno vale 1, e $\log_2 n$ uscite. Il codificatore fornisce in uscita la codifica binaria dell'unica linea di ingresso 1.



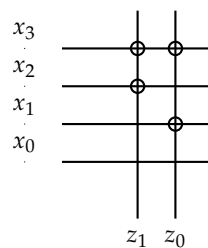
x_3	x_2	x_1	x_0	z_1	z_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Tabella 4.4: Tavola del codificatore

Le equazioni di output saranno:

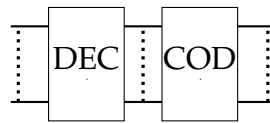
$$z_0 = x_1 + x_3$$

$$z_1 = x_2 + x_3$$



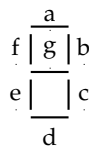
4.2.3 Transcodificatore

Un transcodificatore è formato da un decodificatore e da un codificatore inseriti a cascata. Il transcodificatore avrà quindi n ingressi ed n uscite.



Transcodificatore a sette segmenti

Con il transcodificatore a sette segmenti (chiamato così per il numero di segmenti che formano ogni numero da 0 a 9) è un particolare tipo di transcodificatore che permette di rappresentare un numero binario con la codifica tramite rappresentazione decimale.

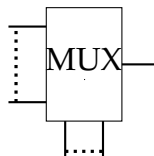


Tramite questo schema possiamo rappresentare tutti i numeri da 0 a 9. Ad esempio il numero 4 sarà formato da b, c, f, g e verrà pertanto rappresentato nel seguente modo:

x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	1	0	0	0	1	1	0	0	1	1

4.2.4 Multiplexer

Il multiplexer è formato da n linee di ingressi di dati, $\log_2 n$ ingressi a selezione (o controllo), di cui solo uno di questi vale 1, e una sola uscita. La funzione fornisce in uscita la linea di dati associata all'unica linea di controllo uguale a 1.



Un MUX è costituito da una porta OR, che riceve le uscite di n porte AND, le quali funzionano da interruttori; infine, un decodificatore attiva l'interruttore selezionato dai segnali di controllo.

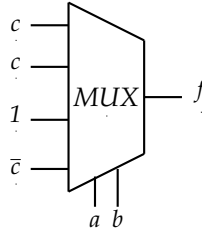
Multiplexer per funzioni booleane

Notiamo come il multiplexer può essere utilizzato per rappresentare le funzioni booleane. Il tutto sarà più chiaro con un esempio.

abc	f
000	0
001	1
010	0
011	1
100	1
101	1
110	1
111	0

Esempio 14. Per rappresentarlo fisso i valori della funzione sugli ingressi e uso le variabili come segnali di controllo. Ottenendo così in ingresso una sequenza di 0 e 1: 01011110, in controllo le tre variabili a, b, c e in uscita la funzione f .

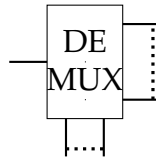
Lo stesso concetto è applicabile rappresentandolo come funzione di 3 variabili ma con un MUX di 4. A differenza di prima avremo quindi come controllo le variabili a, b , come ingresso i 4 valori che identificano nella tabella la funzione quando ad esempio a e b valgono 0, a vale 0 e b vale 1 ... e come uscita la funzione f .



Infatti per a e b presi a due a due nella tabella nei primi quattro casi vale c , nei successivi due vale 1 e negli ultimi due vale $\bar{c}$.

4.2.5 Demultiplexer

Il demultiplexer è formato da una linea di ingresso, $\log_2 n$ linee di controllo, di cui solo una vale 1, e n linee in uscita. La funzione fornisce l'unico ingresso sulla linea di uscita selezionata dall'unica linea di controllo 1.



Un DEMUX è costituito da n porte AND, le quali funzionano da interruttori, e da un decodificatore che attiva l'interruttore selezionato dai segnali di controllo.

Anche in questo caso si possono inserire multiplexer e demultiplexer in cascata.

4.2.6 ROM

Una ROM (Read Only Memory) è un dispositivo con n ingressi (dette linee di indirizzamento) ed m uscite (dette linee dati). All'interno del dispositivo, le linee di indirizzamento selezionano una fra 2^n righe di una matrice $2^n m$. La selezione della riga i -esima della matrice consente di leggere, su ciascuna delle m linee dati, il valore binario memorizzato nella cella. È la composizione di un decodificatore e un codificatore generalizzato cioè di una matrice di AND (le cui entrate sono le linee di indirizzamento e le cui uscite sono le entrate di una matrice di OR) e di una matrice di OR (le cui uscite sono le linee dati).

Il nome ROM (read-only memory) deriva dal fatto che questo modulo combinatorio può essere visto come una memoria (cioè un insieme di celle di dimensione fissata, ognuna con un suo proprio indirizzo) e non riscrivibile (quindi, di sola lettura).

Esempio 15. Data la seguente traccia: rappresenta x in $\mathbb{C}a_2$ da 3 bit con $x \in [-4; 2]$ realizzare la ROM avente output:

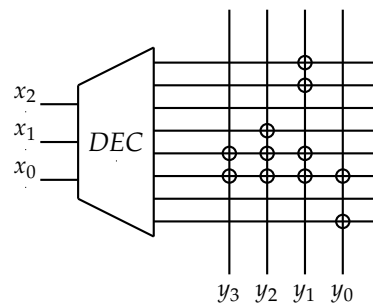
$$y = \begin{cases} 2x & \text{se } x > 0 \\ x + 2 & \text{altrimenti} \end{cases}$$

Ricaviamo la tavola di verità dalla traccia:

La ROM sarà così rappresentata:

	$x_0x_1x_2$	$y_3y_2y_1y_0$
0	000	0010
1	001	0010
2	010	0100
-	011	---
-4	100	1110
-3	101	1111
-2	110	0000
-1	111	0001

Tabella 4.5: Tavola di verità della ROM



4.2.7 PLA

Il PLA o Programmable Logic Array serve per ottenere insiemi di funzioni booleane. E' formato da una matrice di AND, che produce i termini prodotto di funzioni in forma minimale, seguita da una matrice di OR con tante linee quante sono le funzioni da produrre. In tal senso una PLA è più efficiente di una ROM.

Esempio 16. Riprendendo l'Esempio 15, calcoliamo le espressioni minimali con le K-mappe:

x_1x_2	00	01	11	10
x_0				
0	0	0	0	0
1	1	1	0	0

Tabella 4.6: $y_3 = x_2\bar{x}_1$

x_1x_2	00	01	11	10
x_0				
0	0	0	-	1
1	1	1	0	0

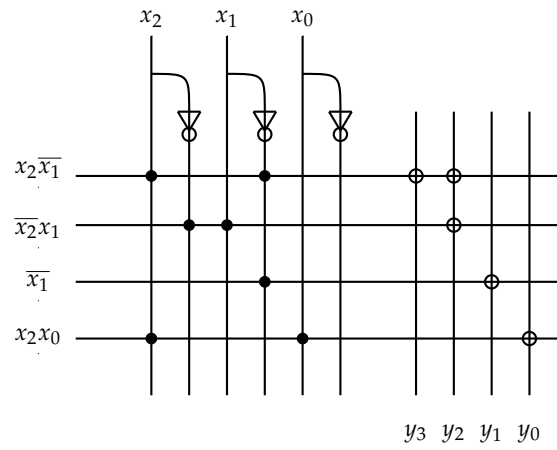
Tabella 4.7: $y_2 = x_2\bar{x}_1 + \bar{x}_2x_1$

x_1x_2	00	01	11	10
x_0				
0	0	0	-	0
1	0	1	1	0

Tabella 4.9: $y_0 = x_2x_0$

Rappresentiamo ora il PLA:

$x_1x_2 \backslash x_0$	00	01	11	10
0	1	1	-	0
1	1	1	0	0

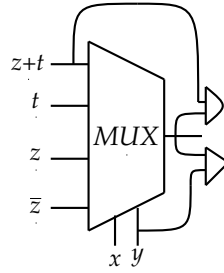
Tabella 4.8: $y_1 = \bar{x}_1$ 

Per approfondire vedi Esercizio 7

4.3 Esercizi svolti

Esercizio 7. Dato il seguente multiplexer trova:

1. Le espressioni di f_1 e f_2 ;
2. La tavola di verità;
3. La ROM;
4. Il PLA.



$$1. f = (z + t)\bar{x}\bar{y} + t\bar{x}y + zx\bar{y} + \bar{z}xy$$

$$f_1 = f(z + t) = (z + t)\bar{x}\bar{y} + t\bar{x}y + zx\bar{y} + (t + \bar{z})xy \text{ Per idempotenza e assorbimento } = \bar{x}\bar{y}z + \bar{x}\bar{y}t + \bar{x}yt + x\bar{y}z + xyt + xy\bar{z} \text{ Elimino i termini non comuni}$$

$$f_1 = \bar{y}z + \bar{x}t + yt + xy\bar{z}$$

$$f_2 = f + y = \bar{x}\bar{y}z + \bar{x}\bar{y}t + \bar{x}yt + x\bar{y}z + xy\bar{z} + y \text{ Applico l'assorbimento } = \bar{x}\bar{y}z + \bar{x}\bar{y}t + x\bar{y}z + y \text{ Elimino i termini non comuni}$$

$$f_2 = \bar{y}z + \bar{x}\bar{y}t + y$$

Rappresentiamo ora le nostre funzioni sulla tavola di verità:

xyzt	f_1	f_2
0000	0	0
0001	1	1
0010	1	1
0011	1	1
0100	0	1
0101	1	1
0110	0	1
0111	1	1
1000	0	0
1001	0	0
1010	1	1
1011	1	1
1100	1	1
1101	1	1
1110	0	1
1111	1	1

Tabella 4.10: Tavola di verità

Con la ROM otteniamo un decodificatore con ingressi x, y, z, y e 16 linee di uscita di f_1 e f_2 che formano la matrice di or.

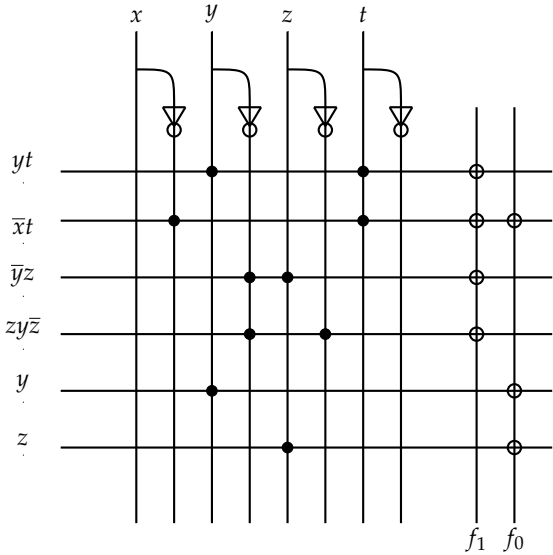
Troviamo le equazioni minimali con le K-mappe e successivamente il PLA:

$xy \backslash zt$	00	01	11	10
00	0	1	1	1
01	0	1	1	0
11	1	1	1	0
10	0	0	1	1

Tabella 4.11: $f_1 = yt + \bar{x}t + \bar{y}z + x\bar{y}z$

$xy \backslash zt$	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	1	1	1	1
10	0	0	1	1

Tabella 4.12: $f_2 = y + z + \bar{x}t$



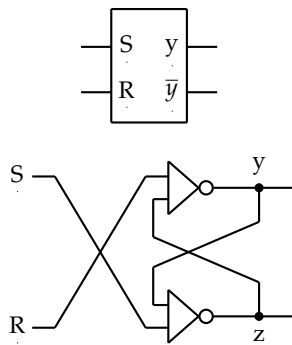
Capitolo 5

Reti sequenziali

Fino ad ora abbiamo progettato circuiti combinatori con l'aiuto dell'algebra booleana. Con circuiti di questo tipo però viene esclusa la possibilità di creare cicli. Vogliamo quindi passare ad altri tipi di circuiti, i cosiddetti circuiti sequenziali, che ci permettono di memorizzare l'informazione attraverso linee di feedback.

5.1 Latch SR

Le prime che analizzeremo sono le latch SR, dove S e R stanno per set e reset. La rappresentazione base di una latch SR è la seguente:



y rappresenta lo stato del latch e il valore del bit memorizzato. Dando dei valori binari ad S e R posso memorizzare o cambiare il valore di y:

1. Per **memorizzare** un valore, devo impostare S e R a 0.

Per capire perchè ciò avviene, "tagliamo" la seconda uscita di y e le estremità le identifichiamo come A e B. L'espressione di y diventa:

$$y = \overline{R + \overline{S + A}} = \overline{R}(S + A)$$

Ora se poniamo $A = 1$ otteniamo $y = 1(0 + 1) = 1$. Essendo $A = B$ notiamo come abbiamo ottenuto lo stesso risultato.

Se poniamo $A = 0$ otteniamo $y = (0 + 0) = 0$. Anche in questo caso $A = B$ quindi possiamo ricongiungere la linea spezzata.

Notiamo come con R e $S = 0$ il valore si mantiene possiamo quindi scrivere la nostra funzione di memorizzazione come:

$$y = \overline{R + z} = \overline{R + (S + y)} = \overline{R}(S + y) \quad (5.1)$$

Può sembrare strano che per calcolare y sto usando ancora una volta y ma dobbiamo tener presente che sono due valori diversi perchè osservati in tempi diversi. Per chiarezza la prima y viene chiamata $y(t+1)$ mentre la seconda (ricavata da z) $y(t)$. $t+1$ è il tempo che mi serve per attraversare le porte NOR.

2. Per **impostare** un valore, devo ottenere $y = 1$ nello stato del latch.

(a) Se $S = 1$ e $R = 0$ stiamo generando una funzione di **set**:

Per $y = 0$ abbiamo che $z = \overline{S + y} = \overline{1 + 0} = 0$ quindi $y = \overline{R + z} = \overline{0 + 0} = 1$.

Per $y = 1$ abbiamo che $z = \overline{S + y} = \overline{1 + 1} = 0$ quindi $y = \overline{R + z} = \overline{0 + 0} = 1$.

Notiamo come in entrambe i casi il valore della y è stato impostato a 1 (anche se nel secondo caso non è visibile).

(b) Con $S = 0$ e $R = 1$ generiamo una funzione di **reset**:

Per $y = 1$ e $z = 0$ abbiamo che $y = \overline{R + z} = \overline{1 + 0} = 0$ e $z = \overline{S + y} = \overline{0 + 0} = 1$.

Per $y = 0$ e $z = 1$ abbiamo che $y = \overline{R + z} = \overline{1 + 1} = 0$ e $z = \overline{S + y} = \overline{0 + 0} = 1$.

In entrambe i casi la y è stata impostata a valore 0.

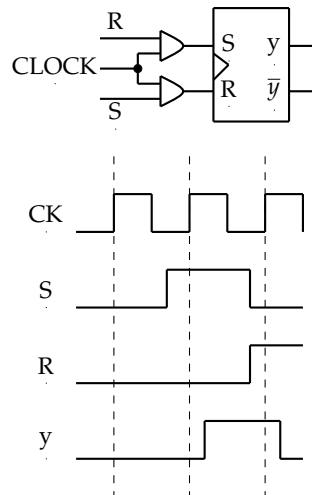
3. Otteniamo una configurazione non ammessa quando sia S che R valgono 1 e la nostra relazione perde così di significato. Questo perchè con y e $z = 0$ si perde la relazione fondamentale $z = \overline{y}$.

SR	$y(t+1)$	Func.
00	$y(t)$	Memorizzazione
01	0	Reset
10	1	Set
11	-	Non ammessa

Tabella 5.1: Tabella del latch SR

Come abbiamo visto quando il segnale passa attraverso delle porte si ha un piccolo ritardo chiamato $\Delta(t)$. Per tanto, nei computer, si utilizza un segnale chiamato clock che ha una sua frequenza e un suo periodo.

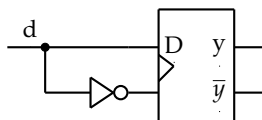
Le latch sono reti sensibili alla temporizzazione. Quando aggiungiamo un latch con un orologio otteniamo un gated latch o **flip-flop**.



Questo accade perchè il cambiamento di S ed R viene catturato sul fronte di salita del clock e successivamente y , con un ritardo $\Delta(t)$, viene impostato al valore corretto.

5.2 Flip-flop D

Il flip-flop di delay deriva dal latch SR e si chiama così perchè il valore di ingresso viene riportato con un certo delay in uscita.

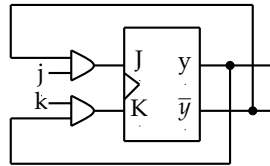


d	y(t+1)
0	0
1	1

In un flip-flop di delay si assume come ingresso solo una variabile d vera e complementata. Con una configurazione del genere, gli ingressi non possono mai essere 0 0 o 1 1, ma si otterranno solo funzioni set o reset.

5.3 Flip-flop JK

Il flip-flop JK deriva dal latch SR.



Ricordando l'Equazione 5.1 avevamo che $y(t+1) = \bar{R}(S + y)$, ma in questa configurazione abbiamo $S = j\bar{y}$ e $R = yk$. Sostituendoli nella y otteniamo: $y(t+1) = (\bar{j}\bar{y} + y)\bar{y}k =$ De Morgan = $y(t+1) = (j\bar{y} + y)(\bar{y} + \bar{k}) =$ assorbimento = $y(t+1) = (j + y)(\bar{y} + \bar{k}) =$ svolgendo i prodotti ottengo $= y(t+1) = j\bar{y} + j\bar{k} + y\bar{y} + y\bar{k} =$

$$y(y+1) = j\bar{y} + j\bar{k} + y\bar{k}$$

A questo punto avremo che se:

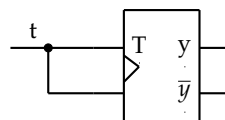
- $j = 0$ e $k = 0$ $y(t+1) = y$
- $j = 0$ e $k = 1$ $y(t+1) = 0$
- $j = 1$ e $k = 0$ $y(t+1) = \bar{y} + 1 + y = 1$
- $j = 1$ e $k = 1$ $y(t+1) = \bar{y}$

jk	y(t+1)	Func.
00	y(t)	Memorizzazione
01	0	Reset
10	1	Set
11	$\bar{y}(t)$	Complemento

Tabella 5.2: Tabella del flip-flop JK

5.4 Flip-flop T

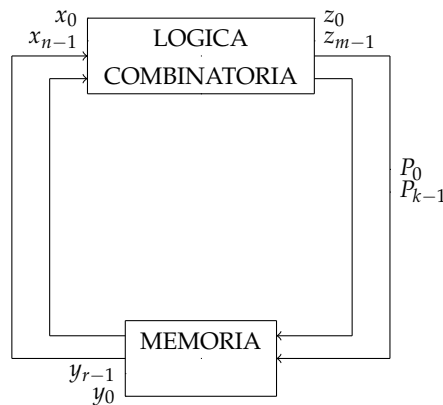
Il flip-flop T (toggle) deriva dal latch SR.



Questo tipo di configurazione può assumere solo i valori $j = 0$ e $k = 0$ o $j = 1$ e $k = 1$.

t	y(t+1)
0	$\bar{y}(t+1)$
1	$y(t+1)$

5.5 Analisi e sintesi di una rete sequenziale



x_0 e x_{n-1} sono gli ingressi, z_0 e z_{m-1} sono le uscite, P_0 e P_{k-1} sono le funzioni di eccitazione.

Le reti sequenziali sono reti più sofisticate dove, oltre alla parte combinatoria, abbiamo anche quella di memoria. Possiamo distinguere, anche per le reti sequenziali, i processi di analisi e sintesi:

- Analisi:** Data una rete usiamo un procedimento per interpretare cosa produce.
- Sintesi:** Data la specifica verbale usiamo un procedimento per progettare la rete minima.

Analisi

Il procedimento per l'analisi di una rete sequenziale è il seguente:

1. Produrre le espressioni booleane delle uscite;
2. Produrre le espressioni booleane delle funzioni di eccitazione;
3. Impostare la tavola degli stati futuri;
4. Costruire il diagramma di stato (automa a stati finiti o macchina a stati finiti);
5. Scrivere la descrizione verbale.

Utilizziamo ora il seguente esempio per capire meglio l'analisi di una rete sequenziale:

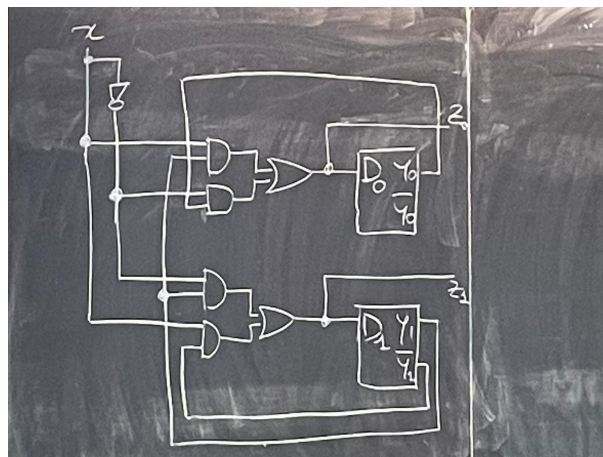


Figura 5.1: Rete sequenziale con Flip-flop di delay

Esempio 17. 1, 2 Scriviamo le espressioni booleane delle uscite e delle funzioni di eccitazione (che in questo caso coincidono).

$$z_0 = xy_1 + \bar{x}y_0 = D_0 \quad z_1 = \bar{x}y_1 + x\bar{y}_1 = D_1$$

3 La tavola degli stati futuri, serve per capire quali saranno i risultati della nostra funzione. E' un modo di rappresentare i dati che abbiamo molto utile perchè ci mostra tutto ciò che può accadere in tutti i casi.

La tavola di compone di quattro colonne. Nella prima troveremo gli ingressi e gli stati rappresentati come combinazioni di valori binari. Nella seconda e terza colonna ci sono rispettivamente le funzioni di eccitazione e di uscita. Nell'ultima colonna rappresentiamo gli stati futuri.

Guardando le espressioni booleane sopra scritte ci chiediamo per quali valori di y_1 e y_0 le funzioni di eccitazione valgono 1? E per quelle di uscita?

xy_1y_0	D_1D_0	z_1z_0	y_1y_0
000	00	00	00
001	01	01	01
010	10	10	10
011	10	11	11
100	10	10	10
101	10	10	10
110	01	01	01
111	01	01	01

Tabella 5.3: Tavola degli stati futuri

Infine avremo tanti stati futuri quanti sono i flip-flop nella rete. I valori degli stati futuri in questo caso sono uguali a quelli degli stati di eccitazione perchè nel flip-flop di delay quando $d = 0$, $y(t+1) = 0$ e quando $d = 1$, $y(t+1) = 1$.

4 Volendo rappresentare in forma grafica la tabella possiamo costruire quello che viene chiamato il diagramma di stato o automa.

Per rappresentarlo dobbiamo scrivere tutti gli stati (tutte le possibili combinazioni di memoria), raffigurati da cerchi etichettati con la configurazione di memoria.

Rappresentiamo poi la transizione tra uno stato e l'altro (passaggio da stato attuale a stato futuro a fronte dei valori di ingresso) tramite delle frecce etichettate da valore di ingresso/valore di uscita.

Seguendo i passaggi il risultato sarà il seguente:

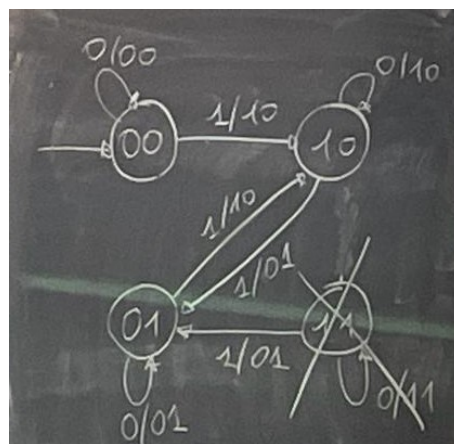


Figura 5.2: Automa a stati finiti

Possiamo infine identificare uno stato di ingresso o eliminare gli eventuali stati che non verranno mai raggiunti.

Per generalizzare il funzionamento della rete, possiamo assegnare dei nomi simbolici agli stati e agli ingressi, in modo da rendere più semplice il processo di descrizione della rete.

Sintesi

Per sviluppare un buon processo di sintesi possiamo seguire questi passi:

- Descrizione verbale;
- Automa della macchina a stati finiti (vedi Sezione 5.5);

- Automa della rete sequenziale (tramite l'assegnamento della codifica binaria a stati, ingressi e uscite);
- Tavola degli stati futuri (vedi Sezione 5.5);
- Espressioni booleane delle funzioni di eccitazione e delle uscite;
- Disegno della rete sequenziale.

Per un'esempio completo, vedi Esercizio 8.

Tabelle inverse dei flip-flop

Per progettare la nostra rete dobbiamo capire quando valgono le funzioni di eccitazione.

Per qualsiasi tipo di flip-flop che consideriamo, il procedimento si svolge sempre allo stesso modo: mi chiedo *passando da y a Y quale funzione o valore ottengo nel flip-flop considerato?*

Ad esempio nel flip-flop SR, passando da 0 a 0 produco le funzioni di memorizzazione e di reset. Produco infatti 0 e δ , in quanto 00 vale per la funzione di memorizzazione e 01 per quella di reset. Si procede in questo modo completando la tabella inversa:

	yY	SR
	00	0 δ
• Flip-flop SR	01	10
	10	01
	11	δ 0

	yY	d
	00	0
• Flip-flop D	01	1
	10	0
	11	1

	yY	jk
	00	0 δ
• Flip-flop JK	01	1 δ
	10	δ 1
	11	δ 0

	yY	t
	00	0
• Flip-flop T	01	1
	10	1
	11	0

Automa a stati finiti

Un'automa a stati finiti (o Finitate State Automata/Machine) rappresenta un'automa con un numero finito di stati. Per descriverlo utilizziamo un'insieme di oggetti:

Descrizione	Simbolo
Insieme finito di stati	Q
Insieme finito di ingressi	ε
Funzione di transizione	δ
Stato iniziale	q_0
Valori di uscita	U
Funzione di uscita	λ

Tabella 5.4: Oggetti dell'automa

Tra gli oggetti rappresentati in Tabella 5.4, abbiamo due funzioni. La prima (funzione di transizione) ci permette di stabilire lo stato futuro nella rappresentazione, dove nel grafico viene espressa tramite una freccia. Essendo una funzione possiamo identificare dominio e codominio:

$$\delta : Q \times \epsilon \rightarrow Q$$

La seconda (funzione di uscita) viene rappresentata tramite l'automa di Mealy o tramite l'automa di Moore a cui fanno riferimento, rispettivamente, le seguenti due funzioni:

$$\lambda : Q \times \epsilon \rightarrow U \quad \lambda : Q \rightarrow U$$

Facciamo un'esempio per capire meglio le differenze tra le due rappresentazioni.

Esempio 18. Ipotizziamo di avere la seguente tabella dell'automa:

	a	b
S_0	S_1/A	S_2/B
S_1	S_0/A	S_3/B
S_2	S_1/B	S_3/A
S_3	S_2/B	S_1/B

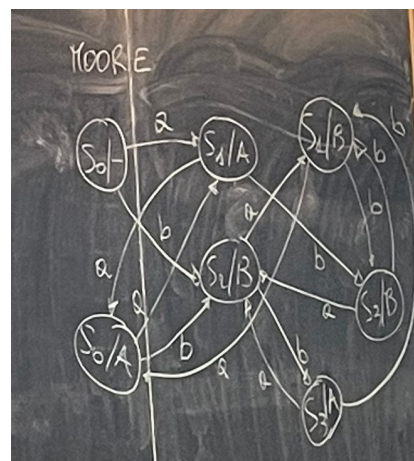
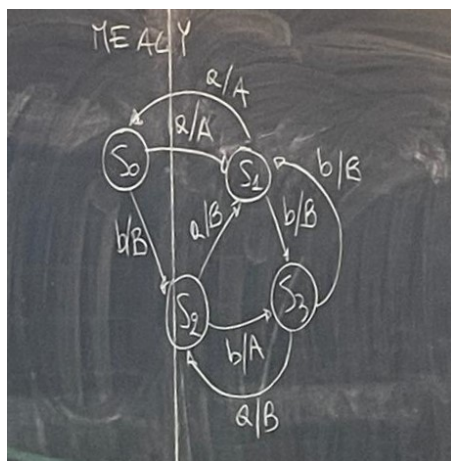


Figura 5.3: Automa di Mealy e Moore

Minimizzare un'automa

Ora che abbiamo capito come produrre degli automi, vediamo come possiamo semplificarli.

Introduciamo quindi due importanti concetti:

- **Equivalenza tra stati**

In un'automa gli stati Q e Q' sono equivalenti se per ogni possibile input, transitano nello stesso stato successivo e producono la stessa uscita.

- **Equivalenza tra automi**

Gli automi A e A' sono equivalenti se per ogni stato di A , $\exists$ uno stato ad esso equivalente nell'automa A' e viceversa.

	a	b	
S_1	$\begin{array}{c} 0 \\ S_2 \end{array}$	$\begin{array}{c} 0 \\ S_6 \end{array}$	
S_2	$\begin{array}{c} 0 \\ S_7 \end{array}$	$\begin{array}{c} 1 \\ S_3 \end{array}$	•
S_3	$\begin{array}{c} 0 \\ S_6 \end{array}$	$\begin{array}{c} 1 \\ S_3 \end{array}$	
S_4	$\begin{array}{c} 1 \\ S_3 \end{array}$	$\begin{array}{c} 0 \\ S_7 \end{array}$	*
S_5	$\begin{array}{c} 0 \\ S_8 \end{array}$	$\begin{array}{c} 0 \\ S_6 \end{array}$	
S_6	$\begin{array}{c} 1 \\ S_3 \end{array}$	$\begin{array}{c} 0 \\ S_7 \end{array}$	*
S_7	$\begin{array}{c} 0 \\ S_7 \end{array}$	$\begin{array}{c} 0 \\ S_5 \end{array}$	
S_8	$\begin{array}{c} 0 \\ S_7 \end{array}$	$\begin{array}{c} 1 \\ S_3 \end{array}$	•

Tabella 5.5: Esempio di equivalenza tra stati di un'automa

Possiamo anche produrre una tabella triangolare dove nella prima colonna sono contenuti tutti gli stati escluso il primo mentre nell'ultima riga troviamo nuovamente tutti gli stati, escluso l'ultimo.

In questo modo possiamo confrontare gli stati a due a due per vedere se sono equivalenti. Se troviamo due stati equivalenti lasciamo la casella vuota, se non sono equivalenti scriviamo una X, mentre scriviamo la coppia degli stati per cui differiscono se le uscite prodotte sono le stesse ma gli stati sono diversi.

S_2	X						
S_3	X	6,7					
S_4	X	X	X				
S_5	2,8	X	X	X			
S_6	X	X	X		X		
S_7	2,7 5,6	X	X	X	7,8 5,6	X	
S_8	X		6,7	X	X	X	X
	S_1	S_2	S_3	S_4	S_5	S_6	S_7

Tabella 5.6: Esempio della tabella triangolare di un'automa

Una volta completata la tabella, dobbiamo controllare l'effettiva equivalenza tra stati: controlliamo se alle coordinate della tabella nei numeri che abbiamo scritto gli stati sono equivalenti. Nel primo caso, per S_1 abbiamo 2,8 controlliamo quindi se la casella 2,8 è vuota. Lo è quindi scriviamo il nome degli stati. Da notare come per i caso successivi 2,7 e 5,6 gli stati non sono equivalenti.

Avremo così ottenuto:

$$S'_1 = \{S_1, S_5\} \quad S'_2 = \{S_2, S_8\} \quad S'_4 = \{S_4, S_6\}$$

Per approfondire vedi Esercizio 9 e 10.

5.6 Esercizi svolti

Esercizio 8. Progettare una rete sequenziale con un solo ingresso x e una sola uscita z tale che $z = 1$ se viene riconosciuta una delle sequenze di ingresso 110 oppure 101 con eventuali sovrapposizioni.

$$x = 00101011000$$

Guardiamo, per ogni bit di x , i due bit che lo precedono. Se la sequenza è uguale a 110 o 101 in z restituiamo 1, altrimenti 0.

$$z = 00001010100$$

Creiamo ora una codifica degli stati in modo tale da identificare ogni coppia di bit con un simbolo.

Codifica degli stati	
Q_{in}	00
Q_1	01
Q_{10}	10
Q_{11}	11

Utilizziamo la codifica appena creata per costruire la tabella dell'automa: Notiamo che lo stato prodotto è

x	0	1
Q_{in}	0 Q_{in}	0 Q_1
Q_1	0 Q_{10}	0 Q_{11}
Q_{10}	0 Q_{in}	1 Q_1
Q_{11}	1 Q_{10}	0 Q_{11}

dato dalla concatenazione dello stato attuale con 0 o 1. Questo produrrà una tripla binaria che ritorniamo come: ultime due cifre della tripla/1 se è la tripla interessata altrimenti 0.

Nel caso in cui otteniamo $Q_{in} + 0 = 000$ che ritorna 00 ovvero Q_{in} e 0. Mentre con $Q_{10} + 1 = 101$ che ritorna 01 = Q_1 e 1.

Possiamo ora rappresentare l'automa a stati finiti:

Per compilare la tavola degli stati futuri guardiamo la tabella dell'automa e compiliamo per x , Y_1 , Y_0 e z :

xy_1y_0	Y_1Y_0	z	d_1d_0
000	00	0	00
001	10	0	10
010	00	0	00
011	10	1	10
100	01	0	01
101	11	0	11
110	01	1	01
111	11	0	11

Possiamo facilmente compilare anche le colonne d_1 e d_0 (abbiamo due flip-flop di tipo d perchè abbiamo due stati futuri) dato che nel flip-flop di tipo d , i valori degli stati di eccitazione equivalgono a quelli degli stati futuri e viceversa.

Osservazione 3. Il flip-flop di tipo d è quello più semplice per questa rappresentazione proprio per quanto descritto sopra.

Non ci resta che calcolare le espressioni booleane minimali con le k -mappe e disegnare la rete sequenziale ottenuta.

$$d_1 = \begin{array}{c|cccc} & y_1y_2 & 00 & 01 & 11 & 10 \\ \hline x & & & & & \\ \hline 0 & 0 & 1 & 1 & 0 \\ \hline 1 & 0 & 1 & 1 & 0 \end{array} = y_0$$

$$d_0 = \begin{array}{c|cccc} & y_1 y_2 & 00 & 01 & 11 & 10 \\ \hline x & & & & & \\ \hline 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{array} = x$$

$$z = \begin{array}{c|cccc} & y_1 y_2 & 00 & 01 & 11 & 10 \\ \hline x & & & & & \\ \hline 0 & 0 & 0 & 1 & 0 & \\ 1 & 0 & 0 & 0 & 1 & \end{array} = \bar{x}y_1y_0 + xy_1\bar{y}_0$$

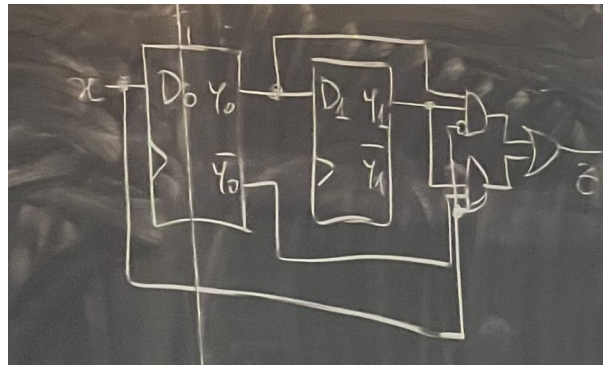


Figura 5.4: Rete sequenziale dell'Esercizio 8

Esercizio 9. Ipotizziamo di avere la seguente tabella dell'automa:

	0	1
A	00 G	01 C
B	00 G	01 D
C	10 D	01 A
D	10 C	01 B
E	00 G	01 F
F	10 F	01 E
G	01 A	11 F

Rappresentiamo in tabella triangolare gli stati e le eventuali equivalenze:

B	CD					
C	X	X				
D	X	X	AB			
E	CF	DF	X	X		
F	X	X	DE,AE	CF,BE	X	
G	X	X	X	X	X	X
	A	B	C	D	E	F

Notiamo come, in questo caso, non trovando stati equivalenti nella tabella triangolare abbiamo bisogno di costruire un cosiddetto grafo delle equivalenze che collega i punti degli stati che presentano delle equivalenze "numeriche".

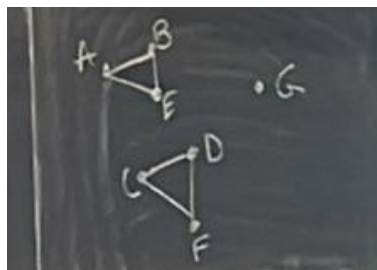


Figura 5.5: Grafo delle equivalenze

Otteniamo così le forme:

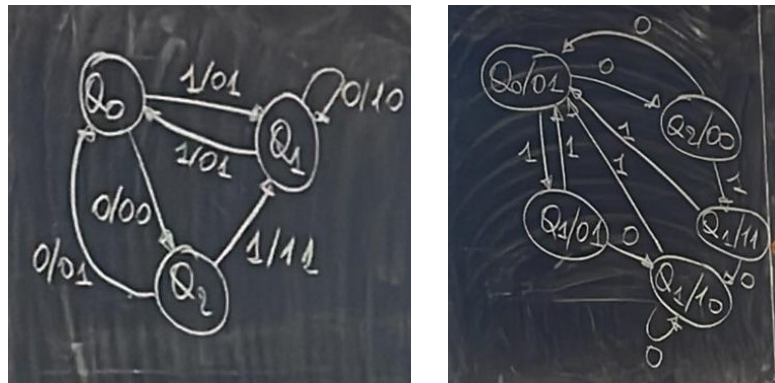
$$Q_0 = A' = \{A, B, E\} \quad Q_1 = C' = \{C, D, F\} \quad Q_2 = G$$

Riscriviamo la tabella dell'automa con il raggruppamento a stati appena svolto:

	0	1
Q ₀	00 Q ₂	01 Q ₁
Q ₁	10 Q ₁	01 Q ₀
Q ₂	01 Q ₀	11 Q ₁

Gli stati e le uscite sono date, come sempre, dal valore che ci viene restituito se a Q₀ passiamo prima 0 e poi 1. Infatti Q₀ segue l'andamento di A,B,E che con input 0 ci portano a G\00 quindi a Q₂\00.

Rappresentiamo infine con gli automi di Mealy e Moore:



Esercizio 10. Si minimizzi il seguente automa e si disegni l'automata minimo sia secondo il modello di Mealy che secondo il modello di Moore.

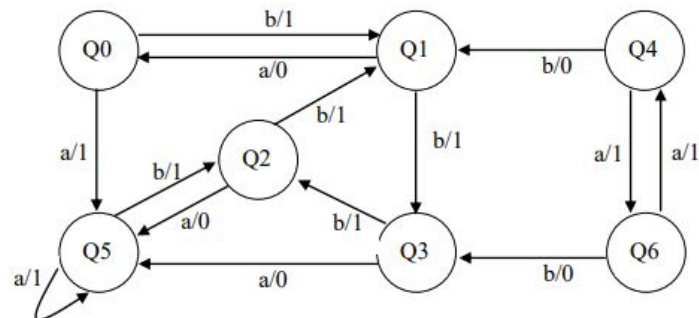


Figura 5.6: Diagramma dell'automata

1. Scrivo la tavola dell'automata:

	a	b
Q ₀	1 Q ₅	1 Q ₁
Q ₁	0 Q ₀	1 Q ₃
Q ₂	1 Q ₅	1 Q ₁
Q ₃	0 Q ₅	1 Q ₂
Q ₄	1 Q ₆	0 Q ₁
Q ₅	1 Q ₅	1 Q ₂
Q ₆	1 Q ₄	0 Q ₃

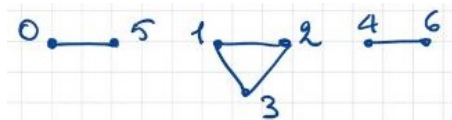
Tabella 5.7: Tavola dell'automata

2. Minimizzo tramite la tabella triangolare:

Q_1	X					
Q_2	X	05,13				
Q_3	X	05,13	12			
Q_4	x	X	X	X		
Q_5	12	X	X	X	X	
Q_6	X	X	X	X	13	X
	Q_0	Q_1	Q_2	Q_3	Q_4	Q_5

Tabella 5.8: Tabella triangolare

Scriviamo sottoforma di grafi gli stati equivalenti:



$$Q'_0 = \{Q_0, Q_5\} \quad Q'_1 = \{Q_1, Q_2, Q_3\} \quad Q'_4 = \{Q_4, Q_6\}$$

3. Rappresentiamo infine la tavola dell'automa minima. Guradando nella tabella di partenza gli stati mi chiedo: in che stato mi portano e che output producono?

	a	b
Q'_0	$\begin{array}{c} 1 \\ Q'_0 \end{array}$	$\begin{array}{c} 1 \\ Q'_1 \end{array}$
Q'_1	$\begin{array}{c} 0 \\ Q'_0 \end{array}$	$\begin{array}{c} 1 \\ Q'_1 \end{array}$
Q'_4	$\begin{array}{c} 1 \\ Q'_4 \end{array}$	$\begin{array}{c} 1 \\ Q'_1 \end{array}$

Tabella 5.9: Tavola minima dell'automa

Esercizio 11. Progettare la rete sequenziale che riceve in ingresso due sequenze di bit x_1 e x_0 e produce due uscite z_1 e z_0 che vengono poste ad 1 in accordo agli ultimi quattro bit ricevuti in ingresso nel modo seguente:

- se nessuno dei bit di ingresso è posto ad 1 oppure un solo bit di ingresso è posto ad 1, allora $z_1 = z_0 = 0$;
- se due qualunque bit di ingresso sono posti ad 1, allora $z_1 = 0$ e $z_0 = 1$;
- se tre qualunque bit di ingresso sono posti ad 1, allora $z_1 = 1$ e $z_0 = 0$;
- se tutti e quattro gli ultimi bit ricevuti in ingresso sono posti ad 1, allora $z_1 = 1$ e $z_0 = 1$.

La rete produce due ulteriori uscite A (allarme) e W (warning) tali che A è posta a 1 se le due uscite z_1 e z_0 sono 1, W è posta a 1 se una delle due uscite z_1 e z_0 è posta a 1 (ma non lo sono entrambe). Si assuma che il primo valore in uscita prodotto per tutti i bit sia 0.

Si realizzi la rete usando un flip-flop di tipo T per il bit più significativo e di tipo JK per il meno significativo. Si realizzino le uscite z_1 e z_0 con ROM e le uscite A e W con porte logiche.

A questo punto possiamo procedere in due modi diversi. Ovvero possiamo assumere che la rete parta con la memoria azzerata (metodo più efficiente) o possiamo partire senza aver ricevuto alcun bit.

1. Disegniamo la tabella dell'automa:

Q_0 bit in memoria 00;

Q_1 bit in memoria 01;

Q_2 bit in memoria 10;

Q_3 bit in memoria 11;

Notiamo come gli stati Q_1 e Q_2 sono equivalenti possiamo pertanto scrivere: $Q'_1 = \{Q_1, Q_2\}$

	x_1x_0 00	x_1x_0 01	x_1x_0 10	x_1x_0 11
Q_0	00 Q_0	00 Q_1	00 Q_2	01 Q_3
Q_1	00 Q_0	01 Q_1	01 Q_2	10 Q_3
Q_2	00 Q_0	01 Q_1	01 Q_2	10 Q_3
Q_3	01 Q_0	10 Q_1	10 Q_2	11 Q_3

Tabella 5.10: Tabella dell'automa

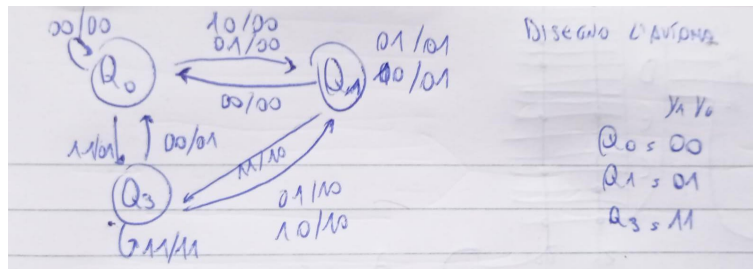


Figura 5.7: Diagramma dell'automa

2. Con le informazioni ottenute dalla tabella dell'automa possiamo rappresentarlo:
3. Scriviamo ora la tavola degli stati futuri, guardando per gli ingressi x_1 e x_0 nella tabella dell'automa e riportando lo stato futuro e l'uscita. Per le funzioni di eccitazione, invece, guardo lo stato attuale e quello futuro e compilo i FF:

$x_1x_0y_1y_0$	Y_1Y_0	z_1z_0	T_1	J_0K_0
0000	00	00	0	0-
0001	00	00	0	-1
0010	-	-	-	-
0011	00	01	1	-1
0100	01	00	0	1-
0101	01	01	0	-0
0110	-	-	-	-
0111	01	10	1	-0
1000	01	00	0	1-
1001	01	01	0	-0
1010	-	-	-	-
1011	01	00	1	-0
1100	11	01	1	1-
1101	11	10	1	-0
1110	-	-	-	-
1111	11	11	0	-0

Tabella 5.11: Tavola degli stati futuri

4. Con le k-mappe troviamo le espressioni minime delle funzioni di eccitazione:

$y_1y_0 \backslash x_1x_0$	00	01	11	10
00	0	0	1	-
01	0	0	1	-
11	1	1	0	-
10	0	0	1	-

$$T_1 = \bar{x}_1y_1 + \bar{x}_0y_1 + x_1x_0\bar{y}_1$$

$$J_0 =$$

$\begin{smallmatrix} y_1 y_0 \\ x_1 x_0 \end{smallmatrix}$	00	01	11	10
00	0	-	-	-
01	1	-	-	-
11	1	-	-	-
10	1	-	-	-

$$= x_0 + x_1$$

$$K_0 =$$

$\begin{smallmatrix} y_1 y_0 \\ x_1 x_0 \end{smallmatrix}$	00	01	11	10
00	-	1	1	-
01	-	0	0	-
11	-	0	0	-
10	-	0	0	-

$$= \overline{x_1 x_0}$$

5. Possiamo ora disegnare il nostro circuito con le specifiche della traccia:

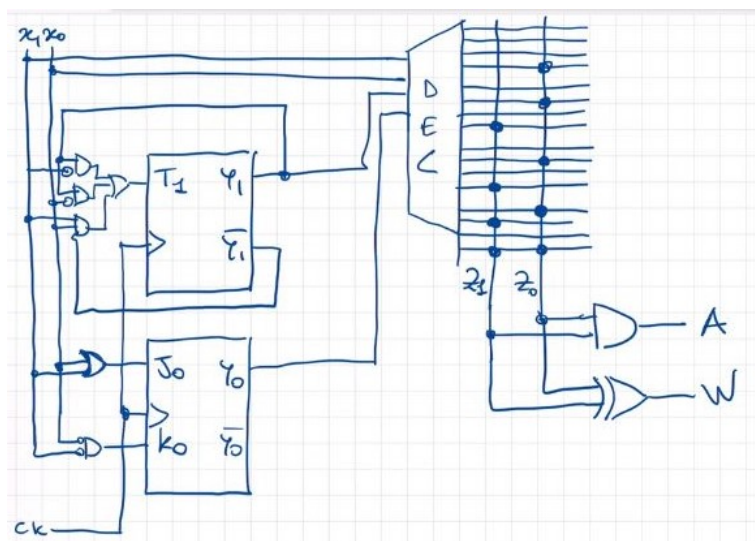


Figura 5.8: Circuito finale

Capitolo 6

Registri

6.1 Contatori

Un contatore è un registro usato per contare il numero di occorrenze di un determinato evento. Tipicamente, gli eventi che può contare sono i colpi di clock o le occorrenze di alcuni valori (o sequenze) di input.

Contatore modulo 8

Il contatore modulo 8 parte da 0 e ad ogni fronte d'onda discendente del clock incrementa di 1 il suo valore, fino ad arrivare a 7; poi torna a 0 e ricomincia.

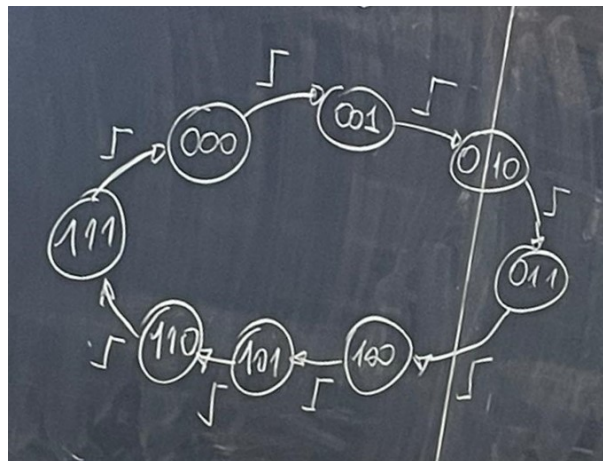


Figura 6.1: Automa contatore modulo 8

Troviamo le equazioni di j e k con le k-mappe:

$$j_0 =$$

y_1y_0	00	01	11	10
y_2	0	1	-	-
1	0	1	-	-

$$j_1 =$$

y_1y_0	00	01	11	10
y_2	-	-	1	0
1	-	-	1	0

$j_2 =$

y_1y_0	00	01	11	10
y_2	0	0	1	0
1	-	-	-	-

Otteniamo così le equazioni:

$$J_0 = K_0 = 1 \quad J_1 = K_1 = y_0 \quad J_2 = K_2 = y_1y_0$$

Con la tavole degli stati futuri otteniamo:

$y_2y_1y_0$	$Y_2Y_1Y_0$	$J_2K_2J_1K_1J_0K_0$
0 0 0	0 0 1	0 - 0 - 1 -
0 0 1	0 1 0	0 - 1 - - 1
0 1 0	0 1 1	0 - - 0 1 -
0 1 1	1 0 0	1 - - 1 - 1
1 0 0	1 0 1	- 0 0 - 1 -
1 0 1	1 1 0	- 0 1 - - 1
1 1 0	1 1 1	- 0 - 0 1 -
1 1 1	0 0 0	- 1 - 1 - 1

Avendo il contatore modulo 2^3 posso rappresentarlo con 3 flip-flop:

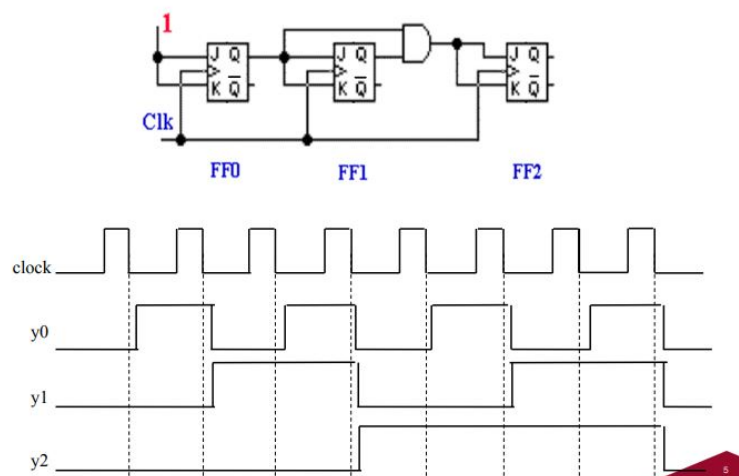


Figura 6.2: Contatore modulo 8 con 3 flip-flop

Contatore bidirezionale

Il contatore bidirezionale è un particolare tipo di contatore che, come suggerisce il termine, può contare crescendo o decrescendo.

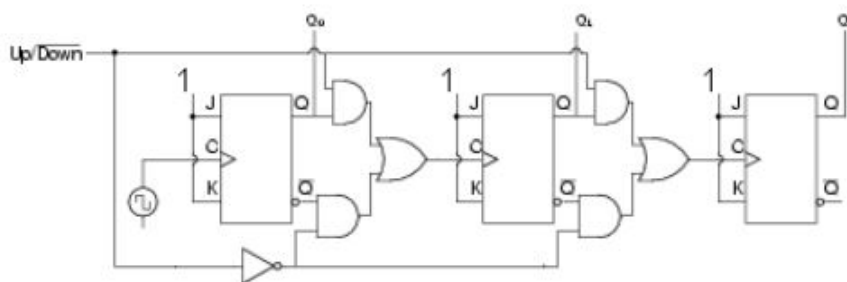


Figura 6.3: Contatore bidirezionale

Se come line di controllo (c) si ottiene 1, il contatore conterà in avanti, in caso contrario, con 0 conta indietro.

Contatore modulo k

Spesso, i flip-flop sono equipaggiati con due ulteriori ingressi, chiamati PRESET e CLEAR, che funzionano in maniera asincrona rispetto al clock: essi cioè sono usati per settare o resettare il flip-flop in modo istantaneo, cioè indipendentemente dalle entrate e dal clock.

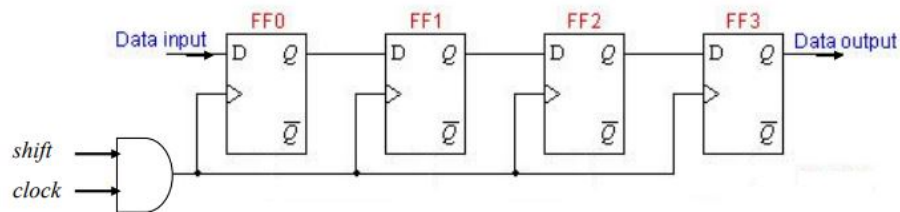


Figura 6.6: Registro SISO

I valori nei registri verranno inseriti in seriale tramite n impulsi di clock, dove n indica il numero di flip-flop. Per caricare n bit, devo fare in modo che shift sia a 1 per n fronti d'onda discendenti del clock e che ad ogni fronte la linea dati entrante contenga il nuovo valore da memorizzare. Per scaricare il registro, bisogna settare shift per n cicli di clock e prelevare i bit dalla linea dati uscente.

Esempio 19. Vogliamo caricare nel registro il valore 1101.

Per la lettura/output si legge il valore nel registro, shiftando i valori verso la linea di uscita seriale, in n cicli di clock.

Tempo	$y_3y_2y_1y_0$
t_0	0000
t_1	1000
t_2	1100
t_3	0110
t_4	1011

Registro SIPO (Serial Input Parallel Output)

La costruzione di un registro SIPO è molto simile a quella del SISO, con la differenza che i valori di lettura/output vengono restituiti in parallelo.

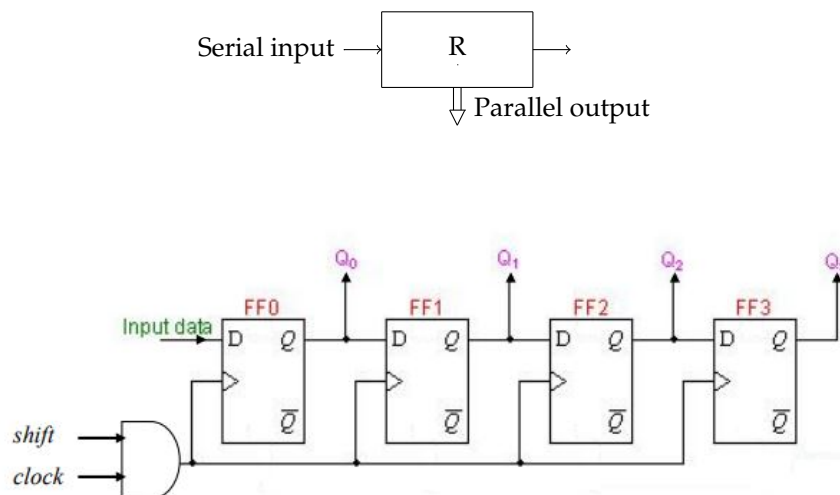


Figura 6.7: Registro SIPO

Registro PISO (Parallel Input Serial Output)

Nei registri con parallel input e serial output abbiamo lo stesso circuito del SISO con la differenza che le entrate: x_0, x_1, x_{n-1} vengono inserite in parallelo e collegate ai flip-flop.

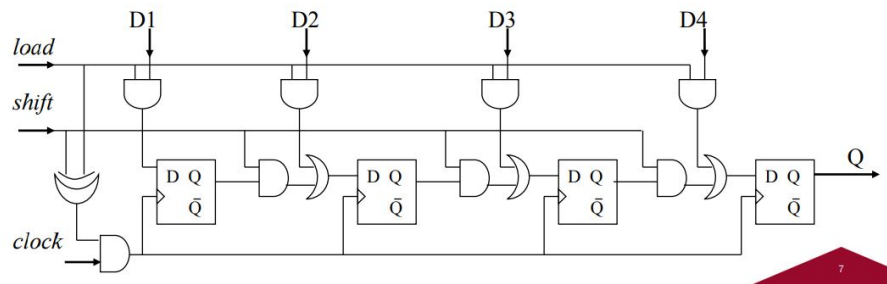


Figura 6.8: Registro PISO

Il registro memorizza gli n input delle linee di dati quando il segnale *load* è a 1 e al momento di un fronte d'onda discendente del clock. Il valore memorizzato viene scaricato quando *shift* vale 1 (in n fronti d'onda del clock).

N.B. Load e shift non possono essere entrambe a 1 nello stesso istante.

Registro PIPO (Parallel Input Parallel Output)

Nei registri con parallel input e parallel output abbiamo lo stesso circuito del SIPO con la differenza che le entrate: x_0, x_1, x_{n-1} vengono inserite in parallelo e collegate ai flip-flop.

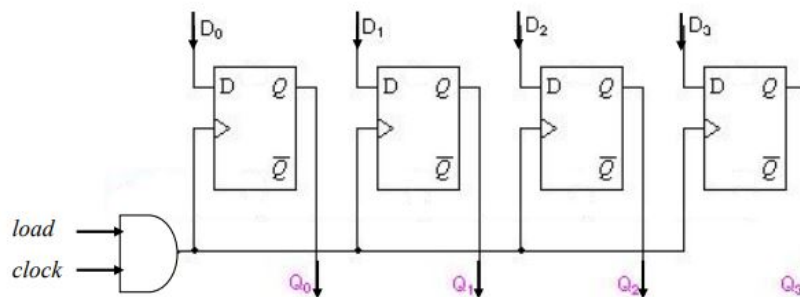


Figura 6.9: Registro PIPO

Il registro riceve in input n linee di dati; tali linee vengono memorizzate quando il segnale *load* è a 1 e al momento di un fronte d'onda discendente del clock. Il valore memorizzato viene riproposto in output dopo t .

Registri a rotazione destra

Nel registro a rotazione a destra i valori vengono fatti ruotare all'interno dei flip-flop. Oltre al serial input troviamo anche il control, ovvero un comando che può assumere valori 0 e 1 e in base a questi decidere se far ruotare i valori o far entrare il serial input.

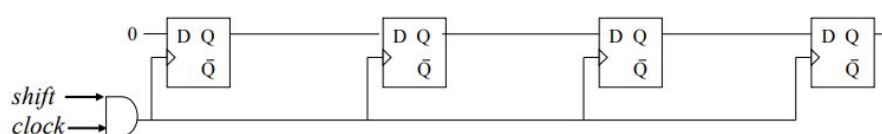


Figura 6.10: Registro a rotazione a destra

Registri a rotazione sinistra

Il funzionamento dei registri a rotazione a sinistra è uguale a quelli con rotazione a destra con l'accortezza che il control e SI vengono spostati all'ultimo flip-flop e i valori vengono appunto shiftati a sinistra.

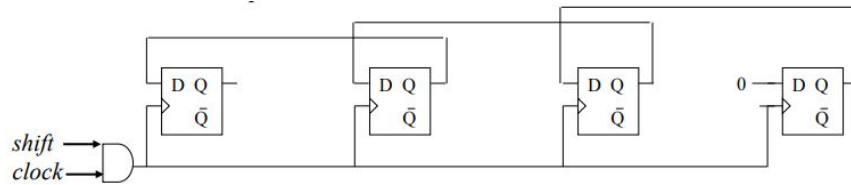
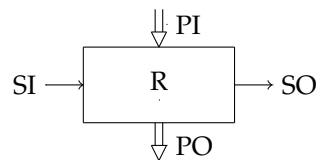


Figura 6.11: Registro a rotazione a sinistra

Registro universale



Nel registro universale abbiamo gli input inseriti in parallelo e seriale e lo stesso per gli output. Inoltre sono presenti le linee di LOAD: car parallelo, RS: right shift, LS: left shift e infine l'enable e il clock.

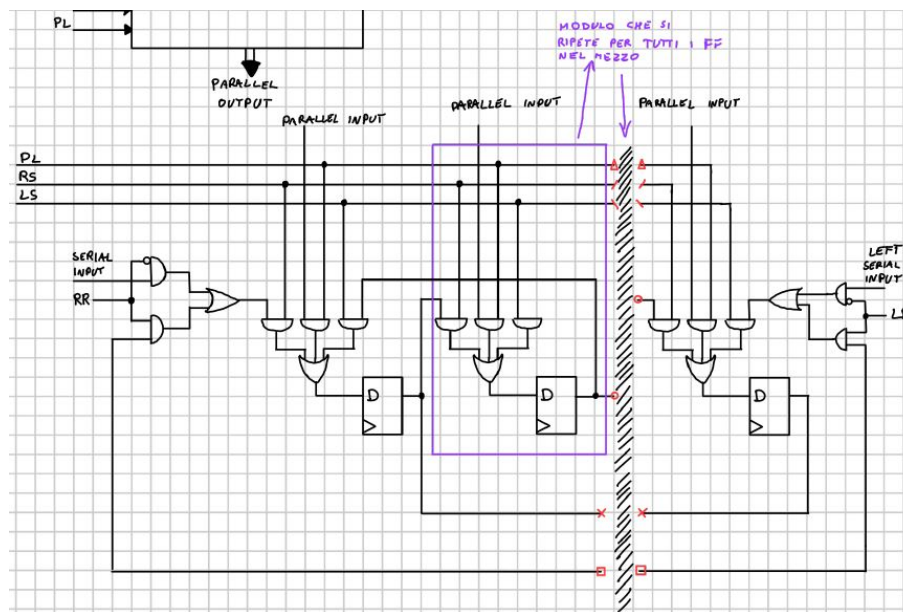


Figura 6.12: Registro universale

6.3 Trasferimento tra registri

Tramite il trasferimento tra registri vogliamo che il contenuto di un registro venga copiato in un registro destinazione. Abbiamo per tanto quattro diverse modalità di trasferimento che dipendono dal numero di registri sorgente e destinazione:

Vediamo ora in maniera schematica i vari casi:

	1 DESTINAZIONE	MOLTE DESTINAZIONI
1 SORGENTE	Porte and Buffer tristate	Decoder
MOLTE SORGENTI	Mux	Mesh (Mux) Bus

Tabella 6.1: Modalità di trasferimento tra registri

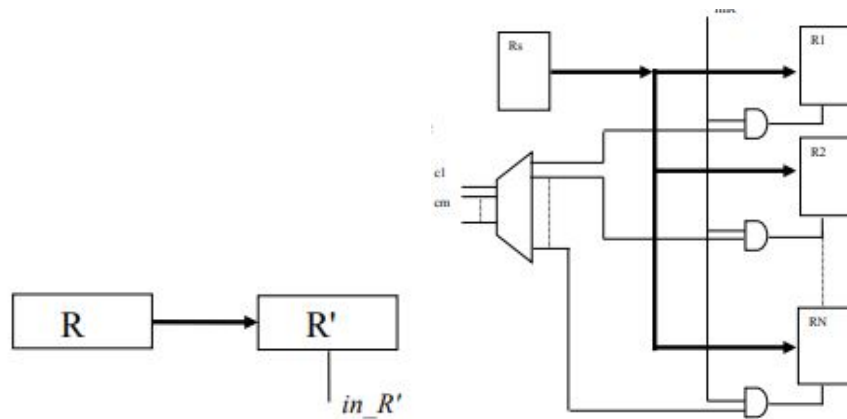


Figura 6.13: Trasferimento con una sorgente e una/molte destinazioni

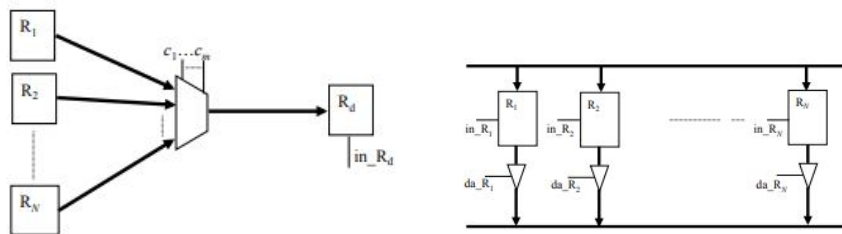


Figura 6.14: Trasferimento con molte sorgenti e una/molte destinazioni

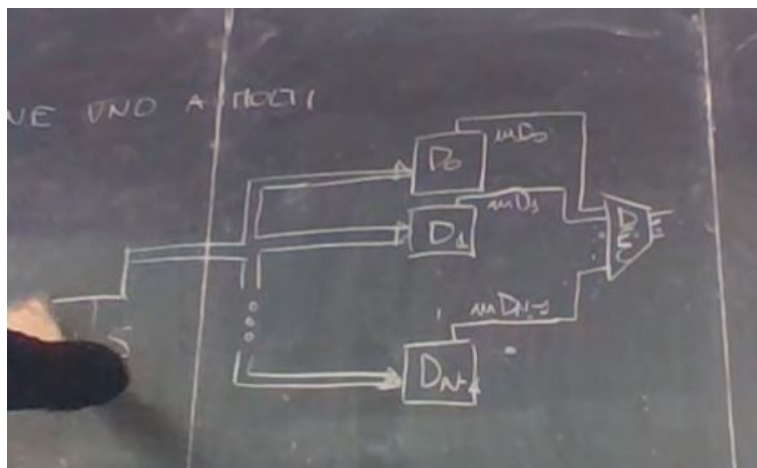
Connessione uno a uno

La connessione 1-A-1 è formata da un'insieme di flip-flop di tipo SR.

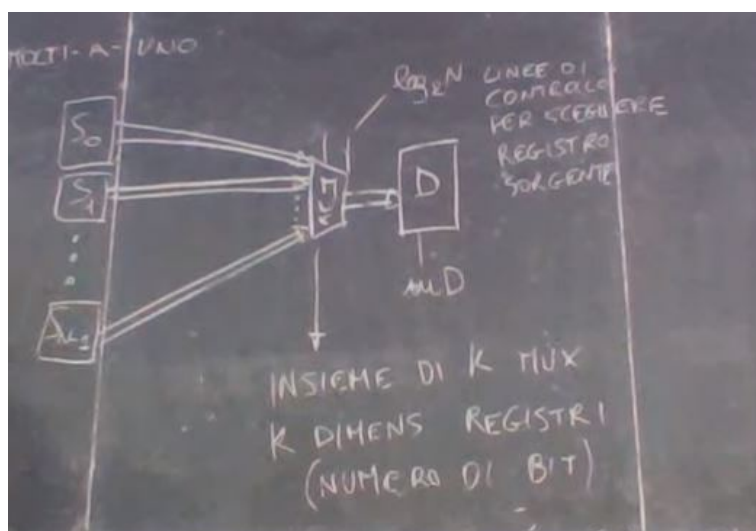


Possiamo usare lo stesso schema per abilitare la lettura da R_5 . In entrambe le rappresentazioni, possiamo usare buffer tristate al posto di porte and.

Connessione uno a molti



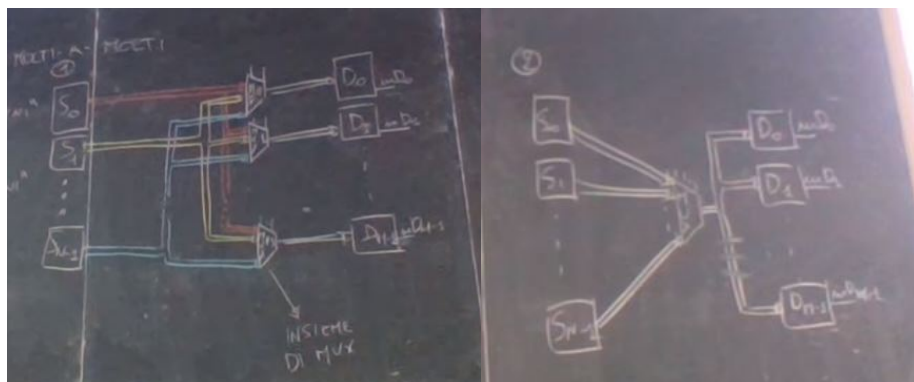
Connessione molti a uno



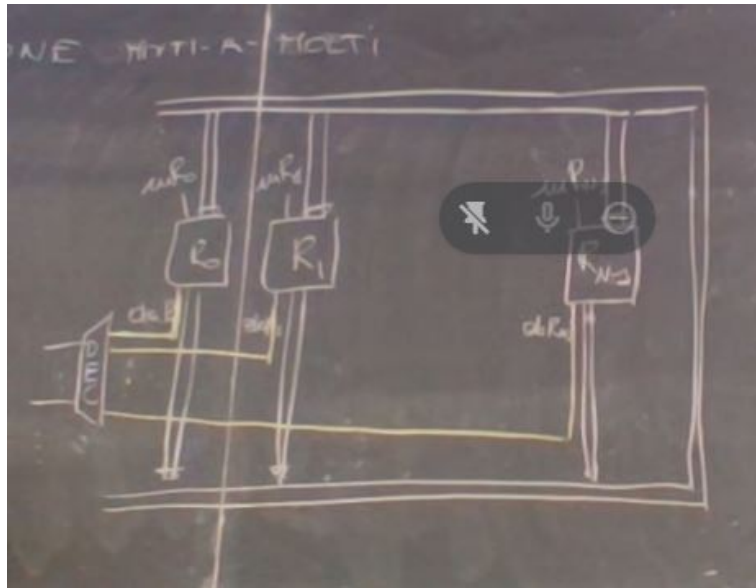
Nelle connessioni molti-a-uno possiamo trovare due costruzioni differenti. Una mesh che si utilizza quando abbiamo registri vicini tra di loro o un bus per quelli lontani.

Connessione molti a molti

Anche in questo caso abbiamo un mesh:



O un bus



6.3.1 Progettare uno schema di connessione tra registri

Per progettare uno schema di connessione tra registri dobbiamo:

- Individuare lo schema per trovare le informazioni;
- Progettare lo schema dei segnali di controllo (come parità(LSB), pos/neg(MSB), multipli di 2^k (LSB));
- Aggiungere i moduli standard (MUX, DEC, ADDER, COMPARATORI).

Per approfondire vedi Esercizio 13.

6.4 Esercizi svolti

Esercizio 12. Sia dato un UP-counter sincrono modulo 4. Si effettui l'analisi (fino all'automa minimo) del seguente circuito sequenziale, inizialmente impostato a $y_0=1$ e $y_1=y_2=0$.

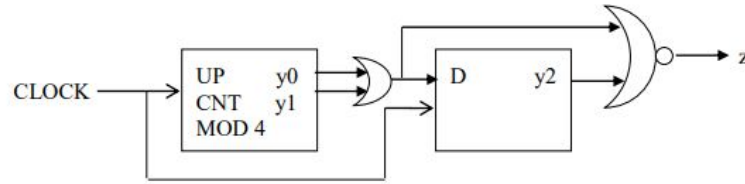


Figura 6.15: Circuito da analizzare

1. Calcolo le espressioni booleane delle funzioni di eccitazione e delle uscite:

$$D = y_1 + y_0 \quad z = \overline{(y_1 + y_0) + y_2} = \overline{y_2 y_1 y_0}$$

2. Scrivo la tavola degli stati futuri. Guardando il nostro circuito 6.15, possiamo compilare lo stato futuro Y_2 tramite y_2 e il FF D, mentre gli stati y_1 e y_0 guardando lo stato successivo perchè ci troviamo in un counter.

$y_2 y_1 y_0$	D	z	$Y_2 Y_1 Y_0$
000	0	1	001
001	1	0	110
010	1	0	111
011	1	0	100
100	0	0	011
101	1	0	110
110	1	0	111
111	1	0	100

Tabella 6.2: Tavola degli stati futuri

3. Scriviamo gli stati Q_n come: $Q_0 = 000$, $Q_1 = 001$, ... $Q_7 = 111$ e stendiamo la tavola dell'automa:

Stato attuale	Stato futuro \ uscita
Q_0	$\begin{array}{c} 1 \\ Q_1 \end{array}$
Q_1	$\begin{array}{c} 0 \\ Q_6 \end{array}$
Q_2	$\begin{array}{c} 0 \\ Q_7 \end{array}$
Q_3	$\begin{array}{c} 0 \\ Q_4 \end{array}$
Q_4	$\begin{array}{c} 0 \\ Q_1 \end{array}$
Q_5	$\begin{array}{c} 0 \\ Q_6 \end{array}$
Q_6	$\begin{array}{c} 0 \\ Q_7 \end{array}$
Q_7	$\begin{array}{c} 0 \\ Q_4 \end{array}$

Tabella 6.3: Tavola dell'automa

Notiamo come gli stati Q_2 e Q_6 , Q_3 e Q_7 , Q_1 e Q_5 sono equivalenti. Possiamo direttamente semplificare l'automa scrivendo:

$$Q'_2 = \{Q_2, Q_6\} \quad Q'_3 = \{Q_3, Q_7\} \quad Q'_1 = \{Q_1, Q_5\}$$

4. Disegniamo ora il diagramma dell'automa sapendo, dalla traccia, che lo stato iniziale è 001 ovvero Q_1 :

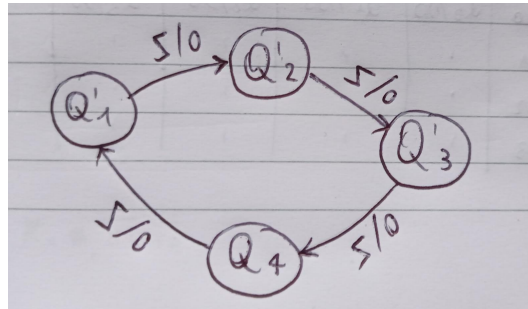


Figura 6.16: Diagramma dell'automa

Esercizio 13. Considerate un insieme di 10 registri R_i (con $i = 0, \dots, 9$) interconnessi tramite bus. Progettare i seguenti trasferimenti:

- Se R_0 è pari R_8 viene trasferito nei registri R_j con $j = 4, \dots, 7$, altrimenti R_7 viene trasferito nei registri R_k con $k = 1, \dots, 3$.
- I trasferimenti sono abilitati se R_9 è multiplo di 4.

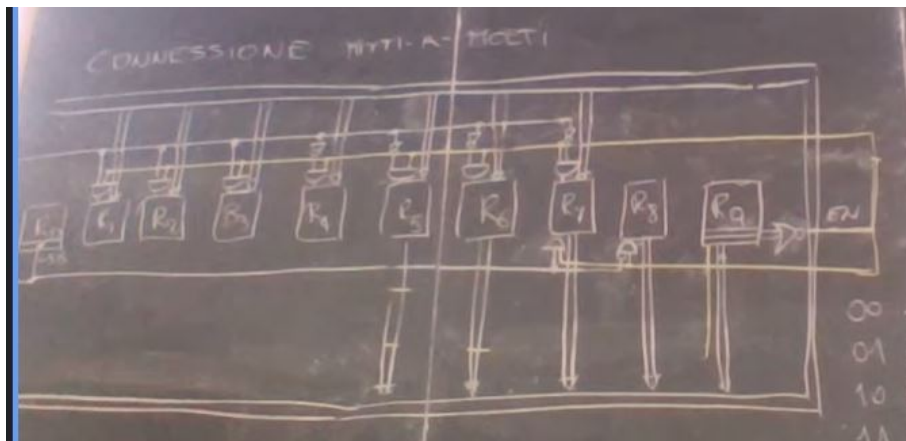


Figura 6.17: Registro risultante

Esercizio 14. Si progetti la rete di interconnessione tale che:

- R_3 viene trasferito in R_0 se R_1 e R_2 sono discordi, in R_1 altrimenti;
- in R_3 viene trasferita la differenza tra R_1 e R_2 se il contenuto di R_0 non è multiplo di 4, la somma tra R_1 e R_2 altrimenti;

Tutti i trasferimenti sono abilitati se R_0 e R_1 sono entrambi pari.

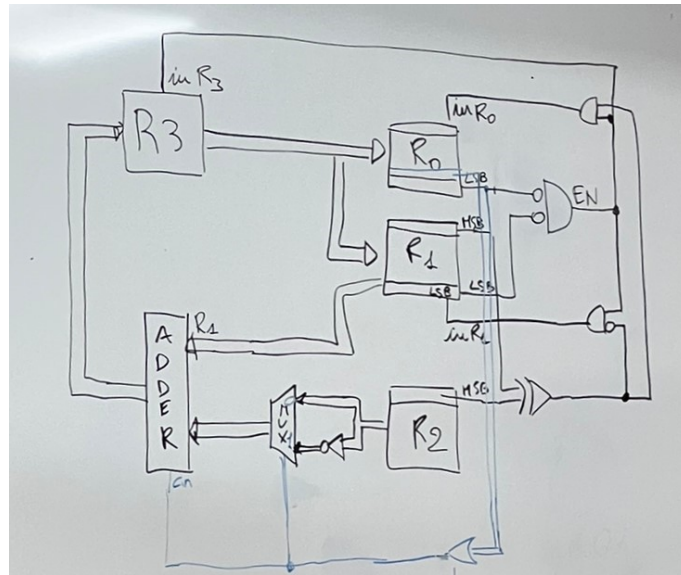


Figura 6.18: Registro risultante

Link utili

- [Convertitore di base](#);
- [Convertitore in virgola mobile](#);
- [Risolutore di espressioni booleane](#);
- [Esercizi di interconnessione tra registri 1](#);
- [Esercizi di interconnessione tra registri 2](#);
- [Esercizi di esame](#) (vedi in Testi compiti scritti).